KOCAELİ ÜNİVERSİTESİ
FEN BİLİMLERİ ENSTİTÜSÜ

YAZILIM MÜHENDİSLİĞİ
ANABİLİM DALI

YÜKSEK LİSANS TEZİ

YAZILIM GELİŞTİRME AKIŞLARINDA BÜYÜK DİL MODELLERİ KULLANARAK HATA ŞİDDETİ TAHMİNİ YAPILMASI

REŞİT DEMİR

KOCAELİ 2026

KOCAELİ ÜNİVERSİTESİ

FEN BİLİMLERİ ENSTİTÜSÜ

YAZILIM MÜHENDİSLİĞİ

ANABİLİM DALI

YÜKSEK LİSANS TEZİ

YAZILIM GELİŞTİRME AKIŞLARINDA BÜYÜK DİL MODELLERİ KULLANARAK HATA ŞİDDETİ TAHMİNİ YAPILMASI

REŞİT DEMİR

Doç. Dr. Hakan GÜNDÜZ		
Danışman,	**Kocaeli Üniv.**	**.....**
Doç Dr. Kaplan KAPLAN		
Jüri Üyesi,	**Kocaeli Üniv.**	**.....**
Dr. Öğr. Üyesi Cumali TÜRKMENOĞLU		
Jüri Üyesi,	**Fatih Sultan Mehmet Vakıf Üniv.**	**.....**

Tezin Savunulduğu Tarih: 20.01.2026

ETİK BEYAN VE ARAŞTIRMA FONU DESTEĞİ

Kocaeli Üniversitesi Fen Bilimleri Enstitüsü tez yazım kurallarına uygun olarak hazırladığım bu tez/proje çalışmasında,

- Bu tezin/projenin bana ait, özgün bir çalışma olduğunu,
- Çalışmamın hazırlık, veri toplama, analiz ve bilgilerin sunumu olmak üzere tüm aşamalarında bilimsel etik ilke ve kurallara uygun davrandığımı,
- Bu çalışma kapsamında elde edilen tüm veri ve bilgiler için kaynak gösterdiğimi ve bu kaynaklara kaynakçada yer verdiğimi,
- Bu çalışmanın Kocaeli Üniversitesi'nin abone olduğu intihal yazılım programı kullanılarak Fen Bilimleri Enstitüsü'nün belirlemiş olduğu ölçütlere uygun olduğunu,
- Kullanılan verilerde herhangi bir tahrifat yapmadığımı,
- Tezin/Projenin herhangi bir bölümünü bu üniversite veya başka bir üniversitede başka bir tez/proje çalışması olarak sunmadığımı,

beyan ederim.

☒ Bu tez/proje çalışmasının herhangi bir aşaması hiçbir kurum/kuruluş tarafından maddi/alt yapı desteği ile desteklenmemiştir.

☐ Bu tez/proje çalışması kapsamında üretilen veri ve bilgiler ..... tarafından ..... no'lu proje kapsamında maddi/alt yapı desteği alınarak gerçekleştirilmiştir.

Herhangi bir zamanda, çalışmamla ilgili yaptığım bu beyana aykırı bir durumun saptanması durumunda, ortaya çıkacak tüm ahlaki ve hukuki sonuçları kabul ettiğimi bildiririm.

.....

Reşit DEMİR

YAYIMLAMA VE FİKRİ MÜLKİYET HAKLARI

Fen Bilimleri Enstitüsü tarafından onaylanan lisansüstü tezimin/projemin tamamını veya herhangi bir kısmını, basılı ve elektronik formatta arşivleme ve aşağıda belirtilen koşullarla kullanıma açma iznini Kocaeli Üniversitesi'ne verdiğimi beyan ederim. Bu izinle Üniversiteye verilen kullanım hakları dışındaki tüm fikri mülkiyet haklarım bende kalacak, tezimin/projemin tamamının ya da bir bölümünün gelecekteki çalışmalarda (makale, kitap, lisans ve patent vb.) kullanımı bana ait olacaktır.
Tezin/projenin kendi özgün çalışmam olduğunu, başkalarının haklarını ihlal etmediğimi ve tezimin/projenin tek yetkili sahibi olduğumu beyan ve taahhüt ederim. Tezimde yer alan telif hakkı bulunan ve sahiplerinden yazılı izin alınarak kullanılması zorunlu metinlerin yazılı izin alarak kullandığımı ve istenildiğinde suretlerini Üniversiteye teslim etmeyi taahhüt ederim.
Yükseköğretim kurulu tarafından yayınlanan ***"Lisansüstü Tezlerin Elektronik Ortamda Toplanması, Düzenlenmesi ve Erişime Açılmasına İlişkin Yönerge"*** kapsamında tezim aşağıda belirtilen koşullar haricinde YÖK Ulusal Tez Merkezi/ Kocaeli Üniversitesi Kütüphaneleri Açık Erişim Sisteminde erişime açılır.

☐ Enstitü yönetim kurulu kararı ile tezimin/projemin erişime açılması mezuniyet tarihinden itibaren 2 yıl ertelenmiştir.
☐ Enstitü yönetim kurulu gerekçeli kararı ile tezimin/projemin erişime açılması mezuniyet tarihinden itibaren 6 ay ertelenmiştir.
☒ Tezim/projem ile ilgili gizlilik kararı verilmemiştir.

.....

Reşit DEMİR

ÖNSÖZ VE TEŞEKKÜR

Yazılım sistemlerinin karmaşıklığının artmasıyla birlikte, yazılım hatalarının erken dönemde tespiti ve doğru şiddet seviyesine göre önceliklendirilmesi yazılım kalite güvencesi süreçlerinin en kritik adımlarından biri hâline gelmiştir. Bu tez çalışmasında, büyük dil modellerinin sunduğu gelişmiş dil ve bağlam anlama yeteneklerinden yararlanarak yazılım hata kayıtlarından hata şiddetini otomatik olarak tahmin etmeye yönelik bir yaklaşım önerilmiştir. Çalışmada, hata kayıtlarının metinsel açıklamaları ile sayısal yazılım metrikleri birlikte değerlendirilmiş ve geliştirilen yöntem gerçek projelerden elde edilen veriler üzerinde sınanmıştır. Elde edilen sonuçların, yazılım bakım süreçlerinde hata önceliklendirme, kaynak planlama ve risk odaklı test faaliyetlerine katkı sağlayacağı değerlendirilmektedir.

Tez çalışmamın her aşamasında bilgi ve deneyimleriyle beni yönlendiren, yapıcı eleştirileri ve sabırlı desteğiyle çalışmanın olgunlaşmasına büyük katkı sunan değerli tez danışman hocam Doç. Dr. Hakan GÜNDÜZ' e içten teşekkür ederim.

Veri hazırlama, deneylerin yürütülmesi ve sonuçların değerlendirilmesi süreçlerinde görüş ve önerileriyle bana destek olan çalışma arkadaşlarıma da ayrıca teşekkür ederim.

Yoğun çalışma dönemlerimde her zaman yanımda olan, sabırları, anlayışları ve manevi destekleriyle beni daima motive eden aileme minnettarlığımı sunarım.

Ocak - 2026 Reşit DEMİR

İÇİNDEKİLER

ETİK BEYAN VE ARAŞTIRMA FONU DESTEĞİ i
YAYIMLAMA VE FİKRİ MÜLKİYET HAKLARI ii
ÖNSÖZ VE TEŞEKKÜR iii
İÇİNDEKİLER iv
ŞEKİLLER DİZİNİ vi
TABLOLAR DİZİNİ vii
SİMGELER VE KISALTMALAR DİZİNİ viii
ÖZET x
ABSTRACT xi
1. GİRİŞ 1
1.1. Araştırma Problemi 3
2. AMAÇ 6
3. HATA TAHMİNİ LİTERATÜR TARAMASI 9
3.1. Geleneksel Hata Tahmini 9
3.2. Hata Tahmininde Derin Öğrenme Tabanlı Yaklaşımlar 10
3.3. Dönüştürücü ve Büyük Dil Modeli Tabanlı Yaklaşımlar 10
4. YÖNTEM VE MİMARİ 12
4.1. Genel Mimari 12
4.2. Veri Kümesi ve Önişleme 16
4.2.1. Ön İşleme Adımı 19
4.3. Özellik Çıkarımı 20
4.3.1. Sayısal Kod Metrikleri 20
4.3.2. Metinsel Temsiller 21
4.3.3. Çok Modlu Birleştirme 21
4.3.4. Lojistik Regresyon 22
4.3.5. Doğrusal Destek Vektör Makineleri (Linear SVM / LinearSVC) 23
4.3.6. Topluluk (Ensemble) Yapıları 23
4.4. Performans Değerlendirme Kriterleri 24
4.4.1. Karışıklık Matrisi (Confusion Matrix) 25
4.4.2. Sınıf Desteği (Support) 25
4.4.3. Doğruluk (Accuracy) 25
4.4.4. Kesinlik ve Duyarlılık (Precision, Recall) 25
4.4.5. F1 Ölçütü 26
4.4.6. Makro ve Ağırlıklandırılmış Ortalama 26
4.5. Öğrenme Paradigmaları 27
4.5.1. Sıfır Örnekli (Zero-shot) Çıkarım 27
4.5.2. Az Örnekli (Few-shot) Öğrenme 28
4.5.3. Tam İnce Ayar (Fine-tuning) Senaryoları 28
4.5.4. Özellik Tabanlı Klasik Makine Öğrenmesi Modelleri 28
4.6. Değerlendirme ve Analiz 29
5. DENEYSEL DÜZENEK 30
5.1. Çalışma Ortamı 30
5.2. Değerlendirme Ölçütleri 31
6. DENEYSEL SONUÇLAR VE TARTIŞMA 32
6.1. Zero-Shot Model Sonuçları 32
6.2. Few-Shot Model Sonuçları 34

6.3. Denetimli Makine Öğrenmesi Modellerinin Sonuçları 36
6.4. Eşik Ayarlaması ve Topluluk Modeli Performansı 36
6.5. Zero-Shot ve Denetimli Yaklaşımların Karşılaştırılması 37
6.6. Hata Şiddeti Seviyelerine Göre Tartışma 38
6.7. Araştırma Hedefleri Açısından Değerlendirme 38
7. SONUÇLAR VE ÖNERİLER 40
7.1. Genel Değerlendirme 40
7.2. Model Performansı ve Çıkarımlar 40
7.3. Öneriler ve Gelecek Çalışmalar 45
KAYNAKLAR 47
EKLER 49
KİŞİSEL YAYIN VE ESERLER 52
ÖZGEÇMİŞ 53

ŞEKİLLER DİZİNİ

Şekil 4.1 Yazılım hata tahmin modeli oluşturma sürecinin şematik gösterimi 13
Şekil 4.2 Veri ön işleme ve akış şeması 16
Şekil 4.3 Çok modlu birleştirme 22
Şekil 4.4 Örnek etiketlerin kullanımı ekran çıktısı 27
Şekil 7.2 İnce ayarlı Codebert ile elde edilen karmaşıklık matrisi 43
Şekil 7.3 Etiket özellik korelasyon matris gösterimi 44

TABLOLAR DİZİNİ

Tablo 4.1 Eğitim ve test veri dosyaları ile örnek sayıları. 17
Tablo 4.2 Çalışmada kullanılan sütunların gösterimi 18
Tablo 6.1 Zero-shot deney sonuçları 33
Tablo 6.2 Zero-shot ve diğer modellerin karşılaştırılması..... 33
Tablo 6.3 Few-shot Sonuçları..... 34
Tablo 6.4 Adım bazlı iyileştirme etkisi. 35
Tablo 6.5 Few-shot modellerin Macro-F1'e göre sıralaması. 35
Tablo 6.6 Model ailesi bazında özet (LogReg / SVM / Ensemble). 35
Tablo 7.1 Test kümesinde modellerin karşılaştırmalı performansı 41
Tablo 7.2 Sınıf bazlı değerlendirme ölçütleri. 45
Tablo 7.3 Karışıklık matrisi (Satır: Gerçek sınıf, Sütun: Tahmin edilen sınıf)..... 45

SİMGELER VE KISALTMALAR DİZİNİ

d	: Derinlik metriği
e	: Kenar (edge) sayısı
fo	: Fan-out metriği
lc	: Belirleyicilik katsayısı
ma	: Maximum McCabe karmaşıklığı
mi	: Maintainability Index (Bakım İndeksi)
ml	: Method length (Metot uzunluğu)
nbd	: Block derinliği
pi	: Package instability (Paket dengesizliği)
r	: Halstead hacmi veya risk metriği (dataset tanımına bağlı)

Kısaltmalar

AI	: Artificial Intelligence (Yapay Zekâ)
ANN	: Artificial Neural Network (Yapay Sinir Ağları)
API	: Application Programming Interface (Uygulama Programlama Arayüzü)
BART	: Bidirectional and Auto-Regressive Transformer (Çift Yönlü ve Otoregresif Dönüştürücü)
BERT	: Bidirectional Encoder Representations from Transformers (Çift Yönlü Transformer Kodlayıcısı)
BiLSTM	: Bidirectional Long Short-Term Memory (Çift Yönlü Uzun Kısa Süreli Bellek)
CLS	: Classification Token (Sınıflandırma Belirteci)
CNN	: Convolutional Neural Network (Evrişimsel Sinir Ağı)
CSV	: Comma-Separated Values (Virgülle Ayrılmış Değerler)
EDA	: Exploratory Data Analysis (Keşifsel Veri Analizi)
F1	: F1-Score (F1 Skoru)
FN	: False Negative (Yanlış Negatif)
FP	: False Positive (Yanlış Pozitif)
GBM	: Gradient Boosting Machine (Gradyan Artırmalı Model)
GPT	: Generative Pre-trained Transformer (Üretici Ön Eğitilmiş Dönüştürücü)
GPU	: Graphics Processing Unit (Grafik İşlem Birimi)
LDA	: Linear Discriminant Analysis (Doğrusal Ayırtman Analizi)
LLM	: Large Language Model (Büyük Dil Modeli)
LSTM	: Long Short-Term Memory (Uzun Kısa Süreli Bellek)
LR	: Logistic Regression (Lojistik Regresyon)
MCC	: Matthews Correlation Coefficient (Matthews Korelasyon Katsayısı)
ML	: Machine Learning (Makine Öğrenmesi)
MLP	: Multi-Layer Perceptron (Çok Katmanlı Algılayıcı)
MSE	: Mean Squared Error (Ortalama Kare Hata)
NLP	: Natural Language Processing (Doğal Dil İşleme)
PCA	: Principal Component Analysis (Temel Bileşenler Analizi)
PR	: Precision–Recall (Kesinlik–Geri Çağırım)
RE	: Regularization (Düzenleme)
RNN	: Recurrent Neural Network (Tekrarlayan Sinir Ağı)
ROC	: Receiver Operating Characteristic (Alıcı İşlemsel Karakteristik Eğrisi)

SVM : Support Vector Machine (Destek Vektör Makineleri)
TF-IDF : Term Frequency – Inverse Document Frequency (Terim Frekansı – Ters Belge Frekansı)
TN : True Negative (Doğru Negatif)
TP : True Positive (Doğru Pozitif)

YAZILIM GELİŞTİRME AKIŞLARINDA BÜYÜK DİL MODELLERİ KULLANARAK HATA ŞİDDETİ TAHMİNİ YAPILMASI

ÖZET

Yazılım projelerinin karmaşıklığı arttıkça, hataların manuel olarak sınıflandırılması zaman alıcı bir hal alıyor ve hata yapma riskini önemli ölçüde arttırmaktadır. Hata şiddetinin doğru tahmini, bakım maliyetlerinin düşürülmesi ve kaynakların verimli kullanımı açısından kritik önem taşımaktadır. Bu çalışmada, yazılım hata şiddetini otomatik olarak belirlemek amacıyla büyük dil modelleri ve statik kod metriklerini birleştiren hibrit bir yöntem geliştirilmiştir. Bu araştırmada, gerçek dünya yazılım projelerinden elde edilen verilerden oluşturulan eğitim veri seti ve bağımsız test veri seti kullanılmıştır. Metinsel verilerin işlenmesinde TF-IDF ve CodeBERT tabanlı vektör temsillerinden, yapısal analizde ise karmaşıklık ve bakım yapılabilirlik gibi statik kod metriklerinden yararlanılmıştır. Veri setindeki sınıf dengesizliğini gidermek amacıyla "Critical" sınıfı "High" sınıfı ile birleştirilerek üçlü bir sınıflandırma yapısı oluşturulmuştur. Lojistik Regresyon, Destek Vektör Makineleri ve Gradient Boosting algoritmalarını içeren çok katmanlı bir topluluk (ensemble) mimarisi kurulmuştur. Deneysel analizler sonucunda, metin ve kod özelliklerinin birlikte kullanıldığı hibrit yaklaşımın, yalnızca sayısal metriklere dayalı temel çizgi modele kıyasla Macro-F1 değerini 0,482'den 0,557'ye yükselttiği ve en iyi yapılandırmada (Ensemble_Tuned) Macro-F1'in 0,693 seviyesine ulaştığı gözlemlenmiştir. Özellikle yüksek öncelikli hataların tespitinde önerilen yöntemin etkinliği ortaya konulmuştur. Sonuç olarak bu tez, hata şiddeti tahmini probleminde büyük dil modeli temsilleri ile statik kod metriklerini birleştiren pratik bir hibrit çerçeve sunmaktadır. Çalışmanın katkıları, gerçek dünya veri setleri üzerinde çok kaynaklı özellik öznitelik birleştirme (füzyon) yöntemi ile hata şiddetinin tahmin edilmesi, sınıf dengesizliğini gidermeye yönelik etiket birleştirme stratejisinin uygulanması, farklı algoritmaları bütünleştiren ensemble mimarisiyle yüksek öncelikli hataların tespit performansının artırılması şeklinde özetlenebilir. Bu yaklaşımın, yazılım bakım süreçlerinde hata önceliklendirme yükünü azaltarak daha hızlı ve tutarlı karar mekanizmalarına zemin hazırlayacağı öngörülmektedir.

Anahtar Kelimeler: Büyük Dil Modelleri, CodeBERT, Hata Şiddeti Tahmini, Hibrit Modelleme, Makine Öğrenmesi.

PREDICTING BUG SEVERITY IN SOFTWARE DEVELOPMENT WORKFLOWS USING LARGE LANGUAGE MODELS

ABSTRACT

As software projects grow in complexity, manually classifying defects becomes time-consuming and significantly increases the risk of human error. Accurate prediction of bug severity is critically important for reducing maintenance costs and using resources efficiently. In this thesis, a hybrid approach is developed to automatically determine software bug severity by combining large language models with static code metrics. The study uses a training dataset and an independent test dataset constructed from data collected from real-world software projects. For processing textual information, TF-IDF and CodeBERT-based vector representations are employed, while structural analysis leverages static code metrics such as complexity and maintainability. To mitigate class imbalance in the dataset, the "Critical" class is merged with the "High" class, resulting in a three-class classification scheme. A multi-layer ensemble architecture is built, integrating Logistic Regression, Support Vector Machines, and Gradient Boosting algorithms. Experimental results show that the proposed hybrid approach, which jointly utilizes text and code features, improves the Macro-F1 score from 0.482 to 0.557 compared to a baseline model relying solely on numerical metrics, and reaches a Macro-F1 of 0.693 under the best configuration (Ensemble_Tuned). The effectiveness of the proposed method is particularly demonstrated in identifying high-priority defects. Overall, this thesis presents a practical hybrid framework for bug severity prediction by integrating large language model representations with static code metrics. The contributions of the study can be summarized as: predicting bug severity on real-world datasets through multi-source feature attribute fusion, applying a label-merging strategy to address class imbalance, and improving detection performance for high-priority defects via an ensemble architecture that combines multiple algorithms. This approach is expected to reduce the burden of bug prioritization in software maintenance processes and support faster, more consistent decision-making mechanisms.

Keywords: Large Language Models, CodeBERT, Defect Severity Prediction, Hybrid Modeling, Machine Learning.

1. GİRİŞ

Yazılım Hatası Tahmini (Software Defect Prediction, SDP) alanındaki çalışmaların temel amacı, hata üretmeye yatkın bileşenleri önceden belirleyerek test ve bakım kaynaklarını en kritik noktalara yönlendirebilmektir (Bersenev ve diğ., 2021). Klasik yaklaşımlar, tasarım karmaşıklığını ve değişiklik riskini yaklaşık olarak ölçebilmek için statik kod metriklerine ve süreç verilerine dayanır. Derin öğrenme ile birlikte alan, kod ve metin üzerinden temsillerin otomatik olarak öğrenildiği modellere kaymış olsa da operasyonel hedef değişmemiştir. Hataları daha erken bularak başarısızlıkları ve maliyetleri azaltmak kritik öneme sahiptir. Tarihsel olarak SDP çalışmaları, programların yapısal özelliklerini yakalayan yazılım metriklerine dayanmaktadır. McCabe, akış grafisi üzerinden tanımladığı çevrimsellik karmaşıklığı ile program karmaşıklığını ölçmeye yönelik bir kontrol akışı metriği önermiş ve bu çalışmasında program karmaşıklığını yönetmeye yönelik bir karmaşıklık ölçütü tanımlamıştır (McCabe, 1976). Daha sonra Chidamber ve Kemerer, nesne yönelimli tasarım için bağlaşıklık ve kavramsal bağlılık gibi kavramları biçimsel olarak tanımlayan CK metrik takımını önererek, bakım yapılabilirlik ve hata eğilimi ile ilişkili temel öznitelikleri sistematik bir biçimde ele almıştır (Chidamber ve Kemerer, 1994). Bu metrikler, sonraki makine öğrenmesi modelleri için hem ortak bir kelime dağarcığı hem de karşılaştırma yapılabilecek temel yöntemler sağlamıştır.

Makine öğrenmesinin yaygınlaşmasıyla birlikte araştırmacılar, SDP problemini metrik vektörleri ve süreç sinyalleri üzerinde çözülen denetimli bir sınıflandırma problemi olarak ele almaya başlamıştır. Sistematik derlemeler, özellik seçimi, topluluk (ensemble) yöntemleri ve dengesiz veriyle başa çıkma teknikleri de dâhil olmak üzere çok geniş bir yöntem yelpazesi ve veri kümesi ailesi sunmakta, özellikle de değerlendirme tasarımının raporlanan kazanımlar üzerinde belirleyici etkisi olduğunu vurgulamaktadır (Alsolai ve Roper, 2022; Pachouly ve Ahirrao, 2022). Bu çalışmalar, adil karşılaştırmalar için veri sızıntısını engelleyen, standartlaştırılmış ve tekrarlanabilir deney düzenlerinin gerekliliğini ortaya koymaktadır.

Büyük ölçekli metin ve kod derlemeleri üzerinde önceden eğitilmiş Transformer tabanlı modellerin ortaya çıkışı, kod ve doğal dilden doğrudan gösterim öğrenmeye imkân tanıyarak alanda önemli bir yöntemsel sıçrama sağlamıştır. CodeBERT gibi kod odaklı büyük dil modelleri, doğal dil ve programlama dili için ortak gömme uzayları

öğrenmektedir. Bu model, programlama ve doğal diller için iki modlu ön eğitimli bir model olarak sunulmakta ve kod arama ile dokümantasyon üretimi gibi aşağı akış görevlerinde güncel başarımlar elde etmektedir (Feng ve diğ., 2020). SDP bağlamında bu tür modeller, isimlendirmeler, kontrol/ veri akışı ipuçları ve kod yazım alışkanlıkları gibi, klasik metriklerin yakalayamadığı anlamsal ipuçlarını modelleyerek, özellikle şiddet odaklı ve anlık (just-in-time) tahmin senaryoları için yeni fırsatlar sunmaktadır.

Buna paralel olarak Just-In-Time (JIT) hata tahmini yaklaşımı, SDP problemini commit (sürüm kaydı) düzeyinde yeniden çerçevelemiştir: bir değişikliğin depoya gönderildiği anda riskli olup olmadığına karar vermek hedeflenir. Güncel bir sistematik inceleme, 67 JIT-SDP çalışmasını derleyerek değişiklik farkları, kod inceleme süreçleri ve geliştirici etkinliği gibi farklı özellik kaynaklarını sınıflandırmakta ve projeler arası genelleme yeteneği yüksek yöntemlere duyulan ihtiyacı vurgulamaktadır (Zhao ve diğ., 2023). JIT ortamında, yalnızca kod farklarına değil aynı zamanda değişiklik özetleri ve commit mesajları gibi çoklu kanıtlara dayanan çok modlu temsillerin önemi öne çıkmaktadır.

SDP alanındaki derin öğrenme odaklı incelemeler, el ile tasarlanmış metriklerden öğrenilmiş temsillere geçişin performans açısından avantajlı olduğunu, ancak veri kalitesi, sınıf dengesizliği ve tekrarlanabilirlik gibi konularda önemli zorluklar doğurduğunu vurgulamaktadır (Bersenev ve diğ., 2021; Alsolai ve Roper, 2022). Uygulamada bu durum, veri bölme stratejilerinin dikkatle tasarlanmasını, ablation deneylerinin yapılmasını ve modellerin özellikle şiddet veya JIT senaryolarına uyarlandığı durumlarda kalibrasyon özelliklerinin raporlanmasını gerektirmektedir. Son yıllara ilişkin bir derleme, yazılım hata tahmininde derin öğrenme tekniklerine odaklandığını belirterek, el ile tasarlanmış özelliklerden veriye dayalı temsillere doğru yaklaşık on yıllık bir dönüşümü özetlemektedir (Bersenev ve diğ., 2021).

Yazılım projelerinde hata şiddeti sınıflandırmasını otomatikleştirmek amacıyla büyük dil modeli tabanlı metinsel temsilleri statik kod metrikleriyle birleştiren çok modlu bir tahmin çerçevesi sunulmuştur. Metinsel alanlar TF-IDF ve CodeBERT tabanlı gömme vektörleri ile temsil edilmiş, sayısal tarafta ise satır sayısı, karmaşıklık ve sürdürülebilirlik göstergeleri gibi kod metrikleri standartlaştırılarak hibrit bir özellik uzayı oluşturulmuştur. Lojistik Regresyon ve LinearSVC gibi denetimli modeller yalnızca metin, yalnızca metrik ve hibrit kurulumlarda eğitilmiş ayrıca CodeBERT, RoBERTa ve

BART-MNLI tabanlı sıfır örneklem, az örneklem ve ince ayar paradigmaları karşılaştırmalı olarak değerlendirilmiştir. Performans doğruluk, makro F1, ağırlıklandırılmış F1, sınıf bazlı ölçütler ve karışıklık matrisi ile raporlanmıştır. Özellikle High ve Critical sınıflarındaki başarı ayrı olarak incelenmiştir. Sonuçlar, metin ve kod metriklerinin birlikte kullanılmasının tek modlu yaklaşımlara göre daha dengeli ve güvenilir tahminler ürettiğini ve kritik şiddet sınıflarının yakalanmasına katkı sağladığını göstermektedir.

Giriş kısmında çalışma motivasyonu ve problem bağlamı sunulmakta, ayrıca temel kavramlar açıklanmaktadır. Bölüm 2'de tezin amacı ve araştırma hedefleri tanımlanmaktadır. Bölüm 3'te geleneksel hata tahmini, derin öğrenme tabanlı yaklaşımlar ve dönüştürücü/LLM tabanlı yöntemler literatür kapsamında incelenmektedir. Bölüm 4'te önerilen yöntem ve mimari, veri kümesi ve önişleme adımları, metinsel ve sayısal özellik çıkarımı, çok modlu birleştirme ve modelleme bileşenleriyle birlikte ayrıntılandırılmaktadır. Ayrıca zero-shot, few-shot, fine-tuning ve klasik makine öğrenmesi paradigmaları açıklanmaktadır. Bölüm 5'te deneysel düzenek, çalışma ortamı ve değerlendirme ölçütleri sunulmuştur. Bölüm 6'da deney sonuçları verilerek yaklaşımlar karşılaştırılmakta ve hata şiddeti seviyelerine göre bulgular tartışılmaktadır. Son olarak Bölüm 7'de genel değerlendirme, çıkarımlar ile gelecek çalışmalara yönelik öneriler paylaşılmaktadır. Ardından kaynaklar, ekler ve özgeçmiş bölümleri yer almaktadır.

1.1. Araştırma Problemi

Sinir ağı tabanlı modellerdeki ilerlemelere rağmen, yazılım hatası tahmini çalışmalarının büyük bir kısmı hâlen el ile tasarlanmış metriklere veya büyük ve etiketlenmiş veri kümelerine bağımlıdır. Ancak etiketli veri kümeleri çoğu zaman sınırlı, projeler arasında dengesiz ve bakımı zordur. Bu tez, özellikle hata şiddetini tahmin etmeye odaklanan bir çerçeve geliştirerek, asgari düzeyde ek etiketleme gerektiren, büyük dil modelleri sıfır örnekli (zero-shot), az örnekli (few-shot) ve ince ayarlı (fine-tuned) yaklaşımlarla hata tahmini problemini ele almaktadır (Mashhadi ve diğ., 2023).

Genel olarak SDP çalışmaları çoğunlukla kodun hatalı olup olmadığını belirlemeye odaklanmakta, bir hatanın üretim ortamına sızması durumunda ne kadar zararlı olacağını

ise çoğu zaman göz ardı etmektedir. Son dönemdeki çalışmaların da vurguladığı gibi, mevcut çalışmaların büyük bölümü tüm hataları aynı önemde kabul etmektedir. Oysa pratikte bu varsayım geçerli değildir ve hata şiddetinin tahmin edilmesi, yüksek etkili hatalara öncelikli kaynak ayrılması için kritik önem taşımaktadır (Mashhadi ve diğ., 2023; Arshad ve diğ., 2024). Bu durum, görevin yalnızca hatanın varlığını işaretlemekle sınırlı kalmadığı, aynı zamanda beklenen etkiyi düşük, orta, yüksek veya kritik gibi seviyelere göre sınıflandırıp sıralayan şiddet odaklı bir tahmin gündemini gerekli kılmaktadır.

Kod tarafında, programlar üzerinde önceden eğitilmiş Transformer tabanlı modeller (örneğin CodeBERT), kaynak kodun sözdizimsel ve anlamsal özelliklerini ortak bir gösterimde toplayarak SDP için umut verici sonuçlar üretmiştir. Bununla birlikte, bu modellerin hata şiddeti odaklı görevler ve heterojen depo yapıları için nasıl özelleştirileceği hâlen açık bir tasarım sorunudur. Pan ve diğ. (2021), CodeBERT modelini yazılım hatası tahmini için kullanmanın uygulanabilirliğini inceleyerek birden fazla tasarım çeşidini karşılaştırmış, ancak hata şiddetine özgü uyarlama, veri açısından verimli ince ayar ve projeler arası genelleme için sistematik bir yol haritasının hâlen eksik olduğunu belirtmiştir (Pan ve diğ., 2021).

Just-In-Time (JIT) senaryolarında, yani riskin commit zamanı tahmin edildiği ortamlarda, topluluk çoğu zaman önceden eğitilmiş kod veya metin kodlayıcılarını kullanmış, ancak ince ayar sürecine ilişkin ayrıntıları sistematik olarak incelememiştir. Guo ve diğ. (2024) tarafından yürütülen ampirik bir çalışma, önceki çalışmaların çoğunda önceden eğitilmiş modellerin doğrudan kullanıldığını, katman dondurma, başlangıç ağırlıkları, öğrenme oranı ve ağırlık çürümesi (weight decay) gibi ince ayar yapılandırmalarının yeterince değerlendirilmediğini vurgulamaktadır. Çalışma, bu ince ayar tercihlerindeki küçük değişimlerin dahi sonuçları anlamlı biçimde etkileyebildiğini göstermekte ve depo yapısındaki zamanla değişim ve sınıf dengesizliği altında dahi kararlılığını koruyabilen, maliyet etkin LLM ince ayar protokollerine (örneğin LoRA tabanlı yaklaşımlar) ihtiyaç olduğunu ortaya koymaktadır (Guo ve diğ., 2024).

Pek çok JIT modeli yalnızca kod farklarına (diff) dayanmakta ve geliştiricinin commit mesajlarında ifade ettiği niyeti göz ardı etmektedir. Jiang ve diğ. (2024) tarafından önerilen iki modlu temsilleme yaklaşımı, mevcut modellerin çoğunun yalnızca kaynak

koddaki düzenlemeleri öğrendiğini, geliştiricinin doğal dilde açıkladığı amaçları dikkate almadığını savunmakta ve kod değişiklikleri ile mesajların birlikte modellenmesinin JIT hata tahmini başarımını iyileştirdiğini göstermektedir. Bu bulgu, kaynak kod, diff, metrikler, commit mesajları ve hata kayıtları gibi çoklu kanıt türlerinin, aşırı uyum (overfitting) ve yorumlanabilirlik kaybı yaratmadan tek bir şiddet odaklı tahmin modelinde nasıl birleştirilebileceği sorusunu gündeme getirmektedir (Jiang ve diğ., 2024).

Ayrıca, derleme çalışmalarında veri gerçekçiliği ve değerlendirme titizliği kronik darboğazlar olarak öne çıkmaktadır. Derin öğrenme tabanlı SDP derlemeleri, literatürdeki çalışmaları analiz ederek etiket gürültüsü, nadir görülen kritik şiddet düzeyleri, veri kümeleri arasındaki seçim yanlılığı ve eğitim/test bölmeleri arasındaki sızıntı gibi açık problemleri tartışmaktadır (Akimova ve diğ., 2021). Bu bulgular, hata şiddeti odaklı LLM yaklaşımlarında dengeli ve sızıntıdan arındırılmış deney düzenlerinin, tekrarlanabilir temel modellerin ve kestirim gücündeki iyileşmenin kullanılan istemler, mimariler ve eğitim stratejileri açısından ayrıştırılabildiği kapsamlı ablation çalışmalarının gerekliliğine işaret etmektedir.

Özetle bu tezde ele alınan araştırma problemi, hata şiddetine odaklanan ve çok modlu bir büyük dil modeli çerçevesi tasarlamaktır. Ayrıca veri açısından verimli ve kararlı ince ayar stratejileri benimsemelidir. Sadece metrik tabanlı ya da tek modlu yaklaşımların ötesine geçip geçmediği de sıkı bir değerlendirme protokolü ile gösterilmelidir. Bu problem tanımı, hata şiddetinin pratikte kritik öneme sahip olmasıyla örtüşür. Aynı zamanda LLM'lerin, el ile oluşturulmuş özelliklerin yakalayamadığı anlamsal örüntüleri modelleyebilmesiyle uyumludur. Bunun yanında araştırma problemi, yazılım hata kayıtlarında yer alan doğal dil açıklamaları ile kod tabanından türetilen yapısal ve sayısal bilgilerin birlikte değerlendirilmesinin sınıflandırma başarımına ne ölçüde katkı sağladığını da ortaya koymayı amaçlamaktadır. Böylece yalnızca doğruluk oranı yüksek bir model geliştirmek değil, aynı zamanda farklı veri temsillerinin hata şiddeti tahminine etkisini sistematik biçimde incelemek hedeflenmiştir. Bu yaklaşım, gerçek dünya yazılım geliştirme süreçlerinde hata önceliklendirme kararlarını destekleyebilecek, geliştirici ekiplerin zaman ve kaynak planlamasını iyileştirebilecek bir karar destek mekanizmasının temelini oluşturması bakımından da önem taşımaktadır.

2. AMAÇ

Bu tez çalışmasının temel amacı, yazılım projelerinde üretilen hata raporu ve değişiklik verilerinden yararlanarak hata şiddetini (ör. düşük–orta–yüksek–kritik) daha tutarlı biçimde tahmin edebilen bir yaklaşım ortaya koymaktır. Uygulamada tüm hataları aynı önemde ele almak mantıklı olmayabilir. Şiddet düzeyi doğrudan iş önceliklendirmesi, müdahale süresi ve kaynak planlaması gibi kararları etkiler (Mashhadi ve diğ., 2023). Bu nedenle çalışma, hata tahminini yalnızca "hatalı/hatasız" ayrımıyla sınırlamak yerine, karar eşikleri ve sınıf öncelikleri operasyonel gereksinimlerle (örneğin SLA ve acil düzeltme kriterleri) uyumlu olacak biçimde tanımlanan çok sınıflı şiddet tahmini problemi olarak ele alır.

Bu ana hedef doğrultusunda ilk amaç, önceden eğitilmiş kod ve metin anlama modellerinin hata şiddeti tahmini bağlamında ne kadar işe yaradığını ölçmektir. Burada yalnızca tek bir yaklaşım seçmek yerine, aynı veri kümesi üzerinde sıfır örnekli (zero-shot), az örnekli (few-shot) ve ince ayarlı (fine-tuning) paradigmaların karşılaştırılması amaçlanır. Böylece, veri miktarının sınırlı olduğu durumlarda hangi yaklaşımın daha uygun olabileceği tespit edilebilir.

İkinci olarak, hata raporlarındaki yapılandırılmamış metin alanlarının (başlık, açıklama, yeniden üretim adımları gibi) dil modelleriyle daha etkin kullanılması hedeflenmiştir. Bu alanlar çoğu zaman geliştiricinin niyetini ve bağlamı metin üzerinden taşır. Dolayısıyla yalnızca sayısal metriklerle yakalanması zor bazı risk işaretleri burada bulunabilir. SevPredict gibi çalışmalar, ince ayarlı dil modellerinin hata raporu metinlerinden anlamlı temsiller çıkarabildiğini ve şiddet tahmini başarımına katkı sağlayabildiğini göstermektedir (Arshad ve diğ., 2024). Bu tez kapsamında da dil modelleri hata raporu alanları üzerinde uyarlanacaktır. Özellikle şiddet bazlı F1 skoru ve olasılık kalibrasyonu gibi ölçütlerdeki etkisi nicel olarak incelenecektir.

Çalışmanın bir diğer amacı, kod merkezli ön eğitimden doğrudan yararlanarak hata sinyallerini yakalamaktır. CodeBERT gibi modeller kaynak kodun sözdizimsel ve anlamsal örüntülerini ortak bir gömü uzayında temsil edebilir ve yazılım tahmini problemlerinde umut verici sonuçlar sunar (Pan ve diğ., 2021). Bu doğrultuda, kodlayıcı tabanlı yaklaşımlar iki şekilde ele alınacaktır. Dondurulmuş kodlayıcı ile özellik çıkarımı

ve seçili katmanların açıldığı ince ayar senaryoları olacaktır. Ayrıca modelin yalnızca tek projede ezber yapmasını önlemek için sürümler arası ve projeler arası genelleme davranışı sistematik biçimde değerlendirilecektir.

Bu tez aynı zamanda çok modlu bir mimari içinde metinsel ve metrik tabanlı özelliklerin bir araya getirilmesini hedefler. Önceki çalışmalar, statik kod metriklerinin CodeBERT benzeri gömülerle birlikte kullanılmasının performansı artırabildiğini göstermektedir (Mashhadi ve diğ., 2023). Bu nedenle büyük dil modeli gömüleri, statik kod metrikleri, mümkün olduğunda commit mesajı ve hata raporu gibi ek metinlerle birlikte kullanılacak, farklı birleştirme stratejileri (erken füzyon, geç füzyon ve kapılayıcı mekanizmaları) karşılaştırılacaktır. Amaç burada sadece daha yüksek skor değildir. Aşırı uyum riskini azaltan ve farklı veri koşullarında daha sağlam kalan bir tasarım bulmaktır.

Bir diğer hedef, Just-In-Time (JIT) senaryolarında commit anında risk tahminini güçlendirecek iki modlu değişim temsilleri geliştirmektir. JIT ortamlarında yalnızca kod farklarına (diff) bakan yaklaşımlar, commit mesajında açıkça ifade edilen niyeti kaçırabilir. Jiang ve diğ. (2024), kod değişiklikleri ile commit mesajlarını birlikte modelleyen iki modlu bir yaklaşımın JIT hata tahmini başarımını artırabildiğini göstermektedir. Kod odaklı temellerle karşılaştırıldığında JIT, F1 ve AUC gibi ölçütlerdeki değişim ortaya konulacaktır.

Bu çalışmanın önemli amaçlarından biri de veri açısından verimli ve kararlı ince ayar protokolleri tasarlamaktır. Guo ve diğ. (2024), literatürde çoğu zaman ince ayar ayarlarının (katman dondurma desenleri, başlangıç ağırlıkları, öğrenme oranı, ağırlık çürümesi vb.) sistematik biçimde raporlanmadığını ve karşılaştırılmadığını vurgulamaktadır. Bu nedenle farklı dondurma desenleri, başlatma stratejileri ve mümkün olduğunda LoRA gibi bellek dostu yöntemler karşılaştırılacaktır. Sınıf dengesizliği, depo ve proje dağılımındaki bozulmalara rağmen performansını koruyan yöntemler belirlenmeye çalışılacaktır.

Ayrıca bu çalışma, veri gerçekçiliği yüksek ve sızıntıdan arındırılmış bir değerlendirme çerçevesi sunmayı amaçlar. Derin öğrenme tabanlı yazılım tahmin çalışmalarında etiket gürültüsü, nadir görülen kritik sınıflar, seçim yanlılığı, eğitim ve test bölmeleri arasındaki sızıntı gibi sorunların sonuçları olduğundan iyi gösterebildiği bilinmektedir (Akimova ve

diğ., 2021). Bu nedenle değerlendirmede dengesiz sınıflara duyarlı ölçütler (makro F1, MCC gibi), sızıntıyı azaltan bölme stratejileri, istem mimari ve eğitim kurallarının katkısını ayırmaya yönelik bileşen çıkarma analizleri (ablation) kullanılacaktır.

Son olarak bu tezde önerilen şiddet odaklı LLM tabanlı yaklaşımlar, klasik makine öğrenmesi ve derin öğrenme temelleri ile karşılaştırmalı olarak değerlendirilecektir. Literatürde erken ve doğru tahminin bakım maliyetlerini düşürebileceği ve güvenilirliği artırabileceği vurgulanmaktadır (Albattah ve diğ., 2024). Bu kapsamda, kabul görmüş ölçütlerle yapılan karşılaştırmalar üzerinden LLM'lerin olası katkısı nicel olarak ortaya konulacaktır. Karşılaştırma sürecinde yalnızca genel başarı oranlarına değil, sınıf bazlı performans farklılıklarına, dengesiz veri yapısının model sonuçlarına etkisine ve özellikle makro ortalamalı ölçütler üzerinden elde edilen bulgulara da dikkat edilecektir. Böylece önerilen yaklaşımın yalnızca baskın sınıflarda değil, daha sınırlı örnek sayısına sahip hata şiddeti düzeylerinde de ne ölçüde tutarlı sonuçlar ürettiği değerlendirilebilecektir. Ayrıca LLM tabanlı yöntemlerin bağlamsal anlamı yakalama, doğal dil ile kod arasındaki ilişkileri birlikte işleyebilme ve özellik çıkarımını daha esnek biçimde gerçekleştirebilme kapasitesi, geleneksel yöntemlerle elde edilen sonuçlar ışığında yorumlanacaktır. Elde edilen bulgular doğrultusunda hangi modelleme yaklaşımının yazılım bakım ve hata yönetimi süreçleri açısından daha uygulanabilir, daha ölçeklenebilir ve karar destek mekanizmalarına daha uygun olduğu da tartışılacaktır.

3. HATA TAHMİNİ LİTERATÜR TARAMASI

Bu bölüm, yazılım hata tahmini disiplininin gelişimini ve hata şiddeti odağındaki literatürü bütüncül bir perspektifle ele almaktadır. Alanın gelişimi incelendiğinde geleneksel metrik tabanlı yaklaşımların zamanla yerini derin öğrenme modellerine, gelinen noktada ise dönüştürücü (transformer) mimarilerin sağladığı yüksek nitelikli temsillere bıraktığı görülmektedir. Bu doğrultuda çalışma, klasik yöntemlerin sunduğu temel yaklaşımlardan başlayarak, derin öğrenmenin getirdiği kapasite artışını ve modern LLM tabanlı çözümlerin şiddet odaklı tahmin senaryolarındaki rolünü sorgulamaktadır. Yapılan bu kapsamlı inceleme, tezin önerdiği hibrit çerçevenin hangi ihtiyaçtan doğduğunu ve mevcut literatürdeki hangi boşluğu doldurduğunu açıkça ortaya koymaktadır.

3.1. Geleneksel Hata Tahmini

SDP alanındaki ilk çalışmalar, çoğunlukla elde hazırlanmış yazılım metriklerine dayanmaktadır. McCabe tarafından önerilen çevrimsel karmaşıklık ölçütü, kontrol akışına dayalı bir karmaşıklık göstergesi olarak yazılan kodun hata içerme olasılığını kabaca tahmin etmeyi amaçlamıştır (McCabe, 1976). Daha sonra Chidamber ve Kemerer, nesne yönelimli sistemler için CK metrik kümesini önererek sınıf başına yöntem sayısı, bağlılık ve uyum gibi özellikleri tanımlamışlardır (Chidamber ve Kemerer, 1994).

Bu metrikler yorumlanabilir ve maliyeti düşük göstergeler sunduğu için klasik makine öğrenmesi modellerinde sıkça kullanılmıştır. Ancak satır sayısı, karmaşıklık ya da sınıf düzeyindeki ölçüler, kodun anlamını, geliştirici niyetini ve hata raporlarındaki bağlamı doğrudan yakalamakta sınırlı kalır. Bu nedenle metrik tabanlı yaklaşımlar çoğu zaman hatalı ile hatasız ayrımına odaklanmıştır. Hata şiddeti gibi operasyonel açıdan önemli bir boyut da çoğu çalışmada geri planda kalmıştır (Akimova ve diğ., 2021).

Bu çalışmada metrikler tamamen bırakılmıyor, ama tek başına da yeterli görülmemiştir. Amaç, metrikleri metin ve kod temsilleriyle birlikte ele alıp şiddet tahminini daha gerçekçi bir zemine taşımak. Böylece hem yorumlanabilirlik korunuyor hem de metriklerin yakalayamadığı bağlamsal ipuçları modele dahil ediliyor.

3.2. Hata Tahmininde Derin Öğrenme Tabanlı Yaklaşımlar

Derin öğrenmenin yazılım mühendisliğine uygulanmasıyla birlikte, kod temsillerini otomatik olarak öğrenmeye çalışan evrişimli sinir ağları (CNN), yinelemeli ağlar (RNN) ve uzun-kısa süreli bellek (LSTM) modelleri geliştirilmiştir. Bu çalışmalar, kaynak kodu veya kod farklarını (diff), karakter, token ya da soyut sözdizim ağacı (AST) dizileri olarak ele alıp, hata tahmini için gözetimli sınıflandırma çerçeveleri önermiştir (Akimova ve diğ., 2021).

Bununla birlikte, derin öğrenme tabanlı erken dönem yaklaşımlar bazı sınırlamalara sahiptir. Öncelikle, çoğu model baştan eğitildiği için büyük ve dengeli etiketli veri kümelerine ihtiyaç duymakta ve bu da projeler arası genelleme yeteneğini kısıtlamaktadır. İkinci olarak, dizisel modeller uzun bağlamları temsil etmekte zorlanmakta ve kodun karmaşık kontrol veri akışı yapılarını tam olarak yakalayamamaktadır. Üçüncü olarak, literatürdeki birçok çalışma, hata tahminini ikili bir problem "hatalı-hatasız" olarak ele almaktadır. Hata şiddeti sınıflandırması çoğu zaman göz ardı edilmektedir (Albattah ve diğ., 2024). SevPredict gibi bazı çalışmalar, hata raporlarının önem derecesini derin öğrenme ve duygu analizi tabanlı özelliklerle tahmin etmeye odaklanmış olsa bile bu modeller genellikle yalnızca metinsel hata raporu verilerini kullanmakta ve kod metrikleri ile kod gömülerini aynı çatı altında birleştirmemektedir (Arshad ve diğ., 2024).

3.3. Dönüştürücü ve Büyük Dil Modeli Tabanlı Yaklaşımlar

Dönüştürücü (Transformer) mimarisi, dikkat (attention) mekanizmaları aracılığıyla uzun bağlamları modelleyebilmesi sayesinde, yazılım hatası tahmini de dahil olmak üzere pek çok yazılım mühendisliği görevinde yeni bir dönemi başlatmıştır. CodeBERT, GraphCodeBERT ve CodeT5 gibi kod odaklı büyük dil modelleri, geniş ölçekli doğal dil ve programlama dili korpusları üzerinde önceden eğitilerek, sözdizimsel ve anlamsal bilgiyi taşıyan bağlama duyarlı gömüler üretmektedir (Feng ve diğ., 2020; Pan ve diğ., 2021). Bu modeller kod sınıflandırma, fonksiyon özetleme, klon tespiti ve hata tahmini gibi görevlerde, geleneksel özellik mühendisliği yöntemlerine kıyasla daha yüksek doğruluk ve F1 skorları rapor etmektedir (Akimova ve diğ., 2021).

Son yıllarda, hata şiddeti tahmini özelinde de büyük dil modellerinin potansiyelini araştıran çalışmalar ortaya çıkmıştır. Mashhadi ve diğ. (2023), yöntem düzeyinde hata şiddeti tahmini için kaynak kod metrikleri ile CodeBERT tabanlı gömüleri birleştiren bir yaklaşım önermekte ve şiddetli hatalar için iyileştirilmiş performans bildirmektedir. Arshad ve diğ. (2024) ise hata raporu metinleri üzerinde ince ayarlanmış bir dil modeli kullanarak hata şiddeti sınıflandırmasında geleneksel makine öğrenmesi yöntemlerine göre daha iyi sonuçlar elde etmektedir. Bununla birlikte, mevcut çalışmaların önemli bir bölümü ya yalnızca hata raporlarına ya da yalnızca kod veya metrik tarafına odaklanmaktadır. Çok modlu, şiddet odaklı ve Just-In-Time senaryoları kapsayan birleşik çerçeveler nispeten az çalışılmıştır (Guo ve diğ., 2024; Jiang ve diğ., 2024).

Bu çalışma, literatürdeki bu boşluğu doldurmayı hedeflemektedir. Projelerinden türetilmiş eğitim veri seti ve test veri seti veri kümelerinde yer alan metinsel alanlar (kod, yorumlar, commit mesajları ve hata raporu açıklamaları) ile sayısal kod metrikleri (örneğin satır sayısı, karmaşıklık ve bakım ölçütleri) birlikte kullanılarak, büyük dil modeli gömüleriyle zenginleştirilmiş çok modlu bir özellik uzayı oluşturulacaktır. CodeBERT ve benzeri modellerin sıfır örnekli, az örnekli ve ince ayarlı sürümleri, geleneksel metrik tabanlı temeller ve tek modlu yaklaşımlar ile karşılaştırılacaktır. Özellikle yüksek ve kritik şiddet düzeylerine ait F1 skorlarında sağlanan kazanımlar ayrıntılı olarak raporlanacaktır.

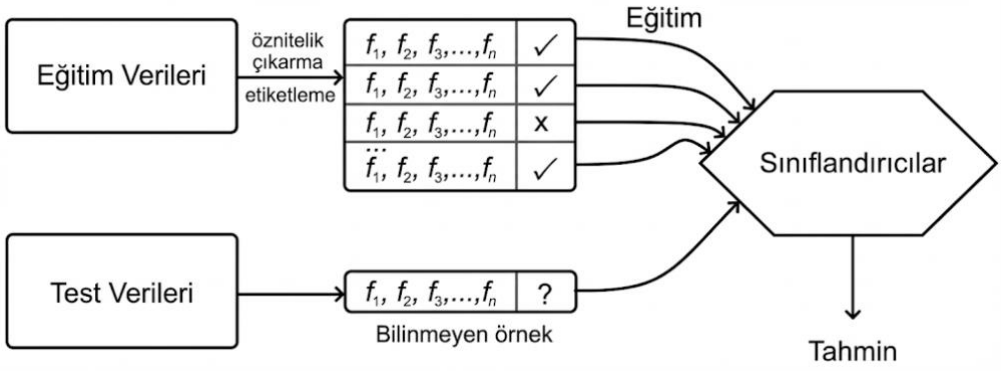
4. YÖNTEM VE MİMARİ

Bu bölümde literatürde incelenen yaklaşımlar temel alınarak, çalışmada izlenen yöntemsel çerçeve ve uçtan uca sistem tasarımı ayrıntılandırılmaktadır. Önceki bölümde özetlenen metrik temelli, derin öğrenme tabanlı ve LLM odaklı yöntemlerin güçlü ve zayıf yönleri dikkate alınarak, hata şiddeti tahmini problemine yönelik hibrit bir çözüm kurgulanmıştır. Bu kapsamda veri kümesinin hazırlanması ve önişleme adımları, metinsel ve sayısal özelliklerin çıkarımı, bu özelliklerin tek bir temsil uzayında birleştirilmesi, kullanılan öğrenme stratejileri ve modelleme bileşenleri ile değerlendirme düzeni bu bölümde sistematik biçimde sunulacaktır. Böylece, önerilen mimarinin hangi gerekçelerle tasarlandığı ve bileşenlerin birbirleriyle nasıl etkileştiği 4.1 Genel Mimari başlığı altında bütüncül bir şekilde açıklanacaktır.

4.1. Genel Mimari

Bu çalışmada, yazılım hatası şiddeti tahmini için büyük dil modeli temsillerini ve geleneksel yazılım metriklerini birlikte kullanan çok modlu bir çerçeve tasarlanmıştır. Önerilen mimari, hata kaydına ait metinsel alanlardan elde edilen dil temsilleri ile kodun yapısal özelliklerini yansıtan sayısal metrikleri aynı karar mekanizmasında birleştirir. Modelin parametreleri eğitim veri seti üzerinden optimize edilmiş, performansı ise bağımsız test veri seti üzerinde raporlanmıştır. Mimari, veri önişleme, özellik çıkarımı, modelleme ve değerlendirme adımlarını ardışık ve tekrarlanabilir bir iş akışı şeklinde kurgular. Böylece, farklı öğrenme paradigmalarının aynı veri bölünmesi üzerinde adil biçimde karşılaştırılması mümkün olur.

Veri katmanı, eğitim veri seti ve test veri seti dosyalarının okunması ve analize hazır hale getirilmesini kapsar. Bu aşamada veri alanları ayrıştırılır ve özellik çıkarımında kullanılacak metinsel ve sayısal sütunlar belirlenmiştir. Metinsel alanlar, hata kaydının açıklaması, kod parçası veya geliştirici notu gibi doğal dil ve kod karışımı içeriği temsil eder. Sayısal alanlar ise satır sayısı, karmaşıklık göstergeleri, bakım indeksi, bağımlılık ilişkileri ve benzeri yazılım metriklerinden oluşur. Veri bütünlüğü kontrol edilmiştir ve eksik veya hatalı kayıtlar analiz dışı bırakılacak biçimde ele alınır. Bu adım, tüm deneylerin aynı veri tanımı üzerinden yürütülmesini sağlamaktadır. Şekil 4.1'de sistemin genel mimarisi görülmektedir.



Şekil 4.1 Yazılım hata tahmin modeli oluşturma sürecinin şematik gösterimi

Özellik çıkarma ve birleştirme katmanı, çok modlu temsilin üretildiği çekirdek bileşendir. Metinsel temsiller iki tamamlayıcı yaklaşımla çıkarılmıştır. Birinci yaklaşımda TF-IDF ile kelime ve karakter düzeyinde seyrek vektörler üretilir. Bu temsil, metin içinde ayırt edici terimlerin ağırlığını yansıtır ve klasik sınıflandırıcılar için güçlü bir temel sağlamaktadır. İkinci yaklaşımda CodeBERT gibi kod odaklı bir Transformer modeli kullanılarak yoğun gömme vektörleri elde edilir. Bu gömmeler, kodun sözdizimsel ipuçlarını ve hata açıklamasındaki bağlamı birlikte taşır. Sayısal kod metrikleri ise StandardScaler ile z-skoru standardizasyonundan geçirilir. Bu işlem, farklı ölçeklerdeki metriklerin model parametrelerini dengesiz biçimde etkilemesini engeller. Sonrasında metinsel özellikler ile standardize metrikler yatay biçimde birleştirilmiştir. Bu birleşim, her hata kaydı için tek bir özellik matrisi oluşturur. Uygulamada SciPy hstack ve scikit-learn Pipeline yapıları ile dönüşümler ardışık ve yeniden üretilebilir biçimde uygulanmıştır.

SciPy hstack: Birden fazla özellik matrisini sütun bazında yan yana ekleyerek tek bir birleşik özellik matrisi oluşturan bir işlemdir. Bu çalışmada metinsel temsiller ile sayısal metrikler aynı satır düzeninde birleştirilmiştir ve modelin tek bir girdi vektörü üzerinden öğrenmesi sağlanmıştır.

TF-IDF: metinleri sayısal bir vektör uzayında temsil etmek için kullanılan yaygın bir özellik çıkarım yöntemidir. TF-IDF, bir terimin ilgili metin içindeki görülme sıklığını (TF) ve terimin tüm veri seti genelindeki ayırt ediciliğini (IDF) birlikte dikkate alarak her terime ağırlık atar. Bu yaklaşımda her terimin ağırlığı, ilgili terimin belgede görülme sıklığını ve tüm koleksiyonda ne kadar ayırt edici olduğunu birlikte dikkate alacak şekilde

hesaplanmıştır. Böylece bir metinde çok geçen ancak birçok belgede de sık görülen genel kelimelerin etkisi azaltılırken, belirli belge veya sınıf için daha ayırt edici terimler daha yüksek ağırlık alır. TF-IDF temsilleri, hata kaydıyla ilişkili metinsel alanlardan kelime ve karakter düzeyinde çıkarılmış ve elde edilen seyrek vektörler, denetimli sınıflandırıcılar için girdi özellikleri olarak kullanılmıştır.

CodeBERT: Microsoft tarafından geliştirilen ve doğal dil metni (ör. hata raporu, açıklama) ile kaynak kodu birlikte anlayacak şekilde önceden eğitilmiş (pre-trained) transformer tabanlı bir dil modelidir. Model, büyük ölçekli kod–metin çiftleri üzerinde eğitildiği için kod parçaları ile bunlara ilişkin açıklamalar arasındaki anlamsal ilişkiyi öğrenir ve bu sayede kod arama, kod özetleme, hata raporu–kod eşleştirme gibi görevlerde güçlü temsil (embedding) vektörlerini üretmiştir.

SVM: TF-IDF temsillerinde etkili bir temel çizgi modeli sağlamaktadır. Gerekli durumlarda SVM skorları kalibrasyonla olasılığa çevrilerek karar eşiği ayarlanmakta ve özellikle kritik sınıflar için hedeflenen duyarlılık düzeyine uygun eşik temelli analiz yapılabilmektedir.

Scikit-learn Pipeline: Veri hazırlama adımlarını ve modeli tek bir iş akışı içinde sıraya koyan bir yapıdır. Ölçekleme, vektörleştirme ve sınıflandırma gibi adımlar tek bir nesne içinde tanımlanır. Bu sayede aynı dönüşümler eğitim ve test verisine tutarlı biçimde uygulanmıştır. Böylece deneysel iş akışı standartlaştırılarak tekrarlanabilirlik güçlendirilmekte, veri sızıntısına yol açabilecek hataların ortaya çıkma olasılığı düşürülmektedir.

Modelleme ve tahmin katmanı, birleşik özellik uzayı üzerinde sınıflandırma modellerinin eğitilmesini ve değerlendirilmesini içerir. Bu tezde iki ana yaklaşım birlikte ele alınır. İlk yaklaşım, özellik tabanlı klasik denetimli öğrenmedir. Burada lojistik regresyon ve doğrusal destek vektör makineleri gibi doğrusal modeller kullanılmıştır. Bu modeller, yüksek boyutlu TF-IDF temsillerinde verimli çalışır ve güçlü bir karşılaştırma zemini sunar. Kalibrasyon ihtiyacı olan senaryolarda SVM skorları olasılığa dönüştürülmüştür ve eşik temelli analiz yapılabilir. İkinci yaklaşım, büyük dil modelleriyle doğrudan çıkarıma dayanan öğrenme paradigmalarıdır. Zero-shot senaryoda model, etiketli eğitim olmadan yalnızca istem cümleleri üzerinden sınıf tahmini üretir. Few-shot senaryoda

sınırlı sayıda örnek, isteme dahil edilerek tahmin davranışı yönlendirilir. Fine-tuning senaryosunda ise model parametreleri görev verisiyle güncellenir. Bu aşamada AdamW gibi optimizasyon yöntemleri, öğrenme oranı planlama ve erken durdurma stratejileri uygulanmıştır. Kaynak kısıtı olan ortamlarda LoRA benzeri düşük-rank uyarlama teknikleri de değerlendirilmiştir. Bu sayede bellek maliyeti azaltılır ve eğitim daha erişilebilir hale gelir.

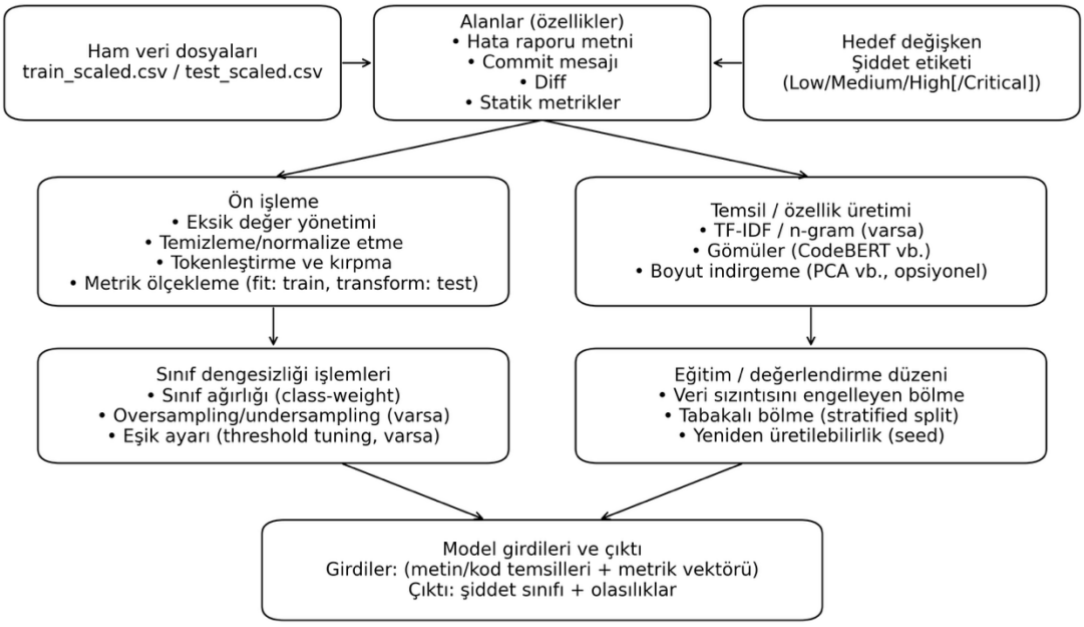
AdamW Optimizasyonu: Derin öğrenme modellerini eğitmek için kullanılan bir optimizasyon yöntemidir. Adam algoritmasının uyarlamalı öğrenme oranı yaklaşımını kullanır. Buna ek olarak ağırlık çürümesini (weight decay) gradyan güncellemesinden ayrı uygular. Bu ayrım, düzenlileştirmenin daha doğru çalışmasını sağlamaktadır. Aşırı uyumu azaltmaya yardımcı olur. Büyük dil modellerinin ince ayarında kararlı yakınsama elde etmek için sık tercih edilir.

Değerlendirme katmanı, tüm modellerin test veri seti üzerinde aynı ölçütlerle raporlanmasını sağlamaktadır. Bu çalışmada doğruluk, makro F1, ağırlıklandırılmış F1, sınıf bazlı precision recall F1 değerleri ve karışıklık matrisi kullanılmıştır. Makro F1, sınıfları eşit ağırlıkla ele aldığı için dengesiz veri koşullarında daha güvenilir bir genel performans göstergesi sunmaktadır. Karışıklık matrisi, sınıflar arası karışmaların yönünü açık biçimde gösterir. Özellikle yüksek şiddet sınıflarında yapılan hata türleri ayrı biçimde incelenmiştir. Böylece yalnızca genel başarı değil, kritik sınıfları yakalama kapasitesi de ölçülmüştür.

Bu çalışmanın temel hedefi, literatürde sık görülen ikili sınıflandırma yaklaşımını aşarak hata şiddetini çok sınıflı biçimde tahmin etmektir. Başlangıçta hedef etiketler Düşük, Orta, Yüksek ve Kritik olarak tanımlanmıştır. Veri dağılım analizi, Kritik sınıfının örnek sayısının çok düşük olduğunu göstermiştir. Bu durum, sınıf dengesizliğini artırmıştır ve modelin azınlık sınıfını öğrenmesini zorlaştırmıştır. Bu nedenle Kritik sınıfı Yüksek sınıfı ile birleştirilmiştir. Sonuçta üç sınıflı bir hedef uzayı elde edilmiştir. Tüm deneyler bu yapı üzerinde yürütülmüştür. Bu karar, değerlendirmede istatistiksel kararlılığı artırmıştır ve sınıf bazlı ölçütlerin daha güvenilir yorumlanmasını sağlamıştır.

4.2. Veri Kümesi ve Önişleme

Kullanılan veri kümeleri, commit düzeyinde oluşturulmuş ve her kayıt için hem metinsel hem de sayısal alanlar içermektedir. Metinsel alanlar, kod gövdesi, kod yorumları, yorumdan arındırılmış kod (code, code_comment, code_no_comment) ve bazı senaryolarda commit mesajı ile hata raporu özetlerini kapsamaktadır. Sayısal alanlar ise satır sayısı (örneğin lc), karmaşıklık, bağımlılık ve bakım ile ilgili çeşitli metriklerden oluşmaktadır. Bu metrikler, model eğitimi sırasında ölçek farklarının etkisini azaltmak amacıyla standartlaştırma ve normalleştirme adımlarına tabi tutulmuştur. Ayrıca, sınıf dengesizliği sorununu azaltmak için eğitim verisi üzerinde yeniden örnekleme ve ağırlıklandırma stratejileri değerlendirilmiş ve seçilen nihai modellerde tutarlı bir etiket dağılımı korunmaya çalışılmıştır. eğitim veri seti dosyası model eğitimi ve doğrulaması için, test veri seti dosyası ise nihai değerlendirme için bağımsız bir test kümesi olarak kullanılmıştır. Bu ayrım sayesinde, modellerin genelleme performansı eğitim sürecinden etkilenmeyen bir veri kümesi üzerinde nesnel biçimde ölçülmüştür. Şekil 4.2'de veri ön işleme aşaması ve uygulanan başlıca teknikler adım adım gösterilmiştir.



Şekil 4.2. Veri ön işleme ve akış şeması

Önişleme adımları kapsamında öncelikle eksik veya tutarsız kayıtlar kontrol edilmiştir. Hedef etiketin bulunmadığı, metinsel alanları tamamen boş olan veya aşırı uç değerler

içeren kayıtlar çalışma kapsamı dışında bırakılmıştır. Kod alanlarında yer alan gereksiz boşluklar, satır sonu karakterleri ve tekrarlayan yorum satırları temizlenmiştir. Buna karşın semantik bilgi taşıyan tanımlayıcı yorumların korunmasına özen gösterilmiştir. Özellikle İngilizce anahtar kelimeler, hata mesajları ve istisna adlarının bozulmaması için agresif bir normalleştirme uygulanmamış, yalnızca temel temizlik adımlarına yer verilmiştir.

Sayısal metrik alanları için, veri kümeleri üzerinde temel istatistikler hesaplanmış ve uç değer analizi gerçekleştirilmiştir. Aykırı değerlerin model öğrenmesini bozmasını engellemek amacıyla, gerekli görülen durumlarda sınır değer kırpma veya log-dönüşümü gibi basit dönüşümler uygulanmıştır. Ardından, tüm metrikler ortalaması sıfır, standart sapması bir olacak şekilde yeniden ölçeklendirilmiş ve böylece metriklerin farklı büyüklük ölçeklerinden kaynaklanan etkileri azaltılmıştır. Bu ölçeklenmiş metrikler, eğitim veri seti ve test veri seti dosyalarında doğrudan kullanılabilmiştir hale getirilmiştir. Bu dosyalardaki her satır, tek bir hata kaydını (veya belirli bir commit ile ilişkili hatayı) temsil etmektedir. Tablo 4.1'de deneylerde kullanılan eğitim ve test veri dosyaları ile bu dosyalardaki sayılar özetlenmektedir.

Tablo 4.1. Eğitim ve test veri dosyaları ile örnek sayıları.

Dosya	**Açıklama**	**Örnek sayısı**
Eğitim veri seti	Eğitim verisi	2414
Test veri seti	Test verisi	502

Metinsel alanlarda, hata ile ilişkili kod ve açıklamaları içeren sütunlar bulunmaktadır:
code: hata ile ilişkili kod parçası.

code_comment: kod içerisindeki yorum satırları.

code_no_comment: yorumları temizlenmiş kod gövdesi.

Bu alanlar hem doğal dil (yorumlar, açıklamalar) hem de programlama dili (kaynak kod) içeren karma metinlerden oluşmaktadır.

Sayısal kod metrikleri ise kodun yapısal özelliklerini nicel olarak ifade eden ve yazılımın karmaşıklık düzeyini, bakım zorluğunu ve genel kalite karakteristiklerini ölçmeye imkân tanıyan sütunlardan oluşmaktadır. Örneğin "lc, pi, ma, nbd, ml, d, mi, fo, r, e" gibi

sütunlar, satır sayısı, yapısal karmaşıklık, iç içe blok derinliği, bakım yapılabilirlik ve modüller arası bağımlılık gibi özellikleri temsil etmektedir. Bu metrikler, yalnızca kodun büyüklüğünü değil aynı zamanda okunabilirliğini, sürdürülebilirliğini ve potansiyel hata üretme eğilimini de dolaylı olarak yansıtmaktadır. Bu sütunlar Tablo 4.2'de detaylı olarak belirtilmiştir.

Tablo 4.2. Çalışmada kullanılan sütunların gösterimi

Kısaltma	Metrik adı	Tanım
lc	Lines Changed (Değişen satır sayısı)	Bir commit/değişiklik sırasında eklenen ve silinen satırların toplamı (LA+LD).
Pi	Prior Issues (Geçmiş hata/issue sayısı)	İlgili dosya/modülün geçmişte ilişkilendirildiği hata veya issue kaydı sayısı.
Ma	Module Age (Modül yaşı)	Dosya/modülün oluşturulmasından veya son büyük değişikliğinden sonraki geçen süre.
Nbd	Number of Bug-fix Changes (Hata düzeltme değişikliği sayısı)	Bileşene dokunan hata düzeltme commit/değişiklik sayısı.
Ml	Module Length / LOC (Modül uzunluğu)	Dosya/modülün toplam kaynak kod satır sayısı (LOC/SLOC).
D	Halstead Difficulty (Zorluk)	Operatör ve operand çeşitliliğine dayalı Halstead zorluk metriği.
Mi	Maintainability Index (Bakım yapılabilirlik indeksi)	LOC, çevrimsellik karmaşıklığı ve Halstead tabanlı ölçütlerden türetilen bakım endeksi.
Fo	Fan-out (Dışa bağımlılık)	Bir modülün çağırdığı veya bağımlı olduğu farklı dış fonksiyon/sınıf/modül sayısı.
R	Ratio (Oran)	Veri çıkarım aracında "r" adıyla raporlanan oran metriği (ör. Yorum oranı, okunabilirlik oranı vb.).
e	Halstead Effort (Çaba)	Halstead hacim ve zorluktan türetilen, kodun geliştirilmesi/anlaşılması için gereken çabayı temsil eden ölçüt.

Veri kümesinde her kayıt için bir hedef değişken (etiket) de bulunmaktadır. Başlangıçta etiketler dört seviyeli bir hata şiddeti yapısını temsil etmektedir (0 = Düşük, 1 = Orta, 2 = Yüksek, 3 = Kritik). Ancak bu çalışma kapsamında, veri kümesindeki dengesiz dağılımı azaltmak ve daha istikrarlı bir öğrenme sağlamak amacıyla kritik sınıfı yüksek sınıfı ile birleştirilmiş ve üç sınıflı bir yapı kullanılmıştır (0 = Düşük, 1 = Orta, 2 = Yüksek/Kritik).

Tüm veriler Python ortamında pandas kütüphanesi kullanılarak okunmakta ve Jupyter Notebook üzerinde analiz edilmektedir. Bu katmanda kullanılan temel yazılım bileşenleri şu şekildedir:

Programlama dili: Python 3.x

Veri analizi: pandas, numpy

Çalışma ortamı: Jupyter Notebook / JupyterLab

Sınıf dengesizliği problemi için, etiket dağılımları incelenmiş ve kritik düzeyde dengesizlik gözlenen senaryolarda ağırlıklandırılmış kayıp fonksiyonları ve katmanlı örnekleme stratejileri kullanılmıştır. Özellikle eğitim aşamasında, sınıf ağırlıkları nadir sınıflar lehine ayarlanarak modelin High sınıfına ait örnekleri ihmal etmesi engellenmiştir. Çalışmada kullanılan özniteliklerin özet istatistikleri ve sınıf dağılımları Ek-A'da sunulmuştur.

4.2.1. Ön İşleme Adımı

Veriler direkt modellere verilmeyip hem metinsel hem de sayısal alanlar üzerinde çeşitli ön işleme adımları uygulanmaktadır. Metinlerin küçük harfe dönüştürülmesi, gereksiz boşlukların ve kontrol karakterlerinin temizlenmesi, bazı özel karakterlerin ayıklanması ve çok kısa veya boş metinlerin elenmesi ya da uygun şekilde işaretlenmesi gibi aşamalar uygulanmaktadır.

Bu adımlar sonucunda, metinler hem TF-IDF gibi klasik yöntemlerle hem de LLM tabanlı gömmelerle (embedding) uyumlu, tutarlı bir forma getirilmektedir.

Sayısal verilerin ölçeklenmesi lc, nbd, mi, fo gibi metriklerin değer aralıkları birbirinden oldukça farklı olabilmektedir. Örneğin, satır sayısı yüzler seviyesinde değişirken, başka bir metrik 0–1 aralığında olabilmektedir. Bu nedenle, scikit-learn kütüphanesindeki StandardScaler gibi araçlar kullanılarak sayısal sütunlar ölçeklenmiştir. Böylece, her metrik yaklaşık olarak ortalaması 0 ve standart sapması 1 olacak şekilde dönüştürülmüştür. Bu adım, özellikle Lojistik Regresyon ve SVM gibi modellerin daha kararlı ve dengeli öğrenmesine katkı sağlamaktadır.

Ön işleme adımları uygulanırken scikit-learn'ün dönüştürücü (transformer) sınıfları kullanılarak ve çoğunlukla Pipeline yapıları içinde bir arada yönetilmektedir. Böylece, eğitim ve test verisi üzerinde aynı işlemler tutarlı bir biçimde uygulanmaktadır.

4.3. Özellik Çıkarımı

Bu bölümde, hata şiddeti tahmininde kullanılacak girdi değişkenleri metinsel ve sayısal bileşenler halinde sistematik olarak çıkarılmaktadır. Amaç, bir hata kaydının hem geliştirici tarafından yazılan açıklama ve kod bağlamını hem de değişikliğin yapısal karmaşıklık ve bakım zorluğuna ilişkin ölçülebilir göstergelerini tek bir temsil altında toplayabilmektir. Bu doğrultuda önce sayısal kod metrikleri standartlaştırılarak modele uygun hale getirilmekte, ardından metinsel alanlardan TF-IDF ve CodeBERT tabanlı gömüler üretilmekte ve son aşamada bu iki kaynak çok modlu bir yapıda birleştirilerek modelleme katmanına aktarılmaktadır. Böylece, tek bir bilgi kaynağına bağımlı kalmadan daha kapsamlı ve karşılaştırılabilir bir özellik uzayı elde edilmesi hedeflenmektedir.

4.3.1. Sayısal Kod Metrikleri

Metinsel özelliklere ek olarak, her hata kaydı için çeşitli kod metrikleri de modele giriş olarak dahil edilmektedir. Bu metrikler, yazılım mühendisliği literatüründe sık kullanılan yapısal göstergeleri içermektedir. Sayısal kod metrikleri, kodun yapısal karmaşıklığını ve bakım zorluğunu yansıtan gösterge değişkenler olarak kullanılmıştır. Veri kümesinde yer alan metrikler arasında satır sayısı, çevrimsel karmaşıklık, fonksiyon veya yöntem sayısı, dallanma sayısı, dışa ve içe bağımlılık sayıları ve bakım indeksine ilişkin ölçütler bulunmaktadır. Bu metrikler, literatürde yaygın olarak kullanılan CK ve McCabe tabanlı metrik aileleri ile uyumludur.

Tüm metrikler, model eğitiminde sayısal kararlılığı sağlamak amacıyla standart skor (z-skoru) ile ölçeklendirilmiştir. Bu metrikler pandas ile veri çerçevesi sütunları şeklinde seçilmekte ve StandardScaler gibi dönüştürücülerle ölçeklenmekte, daha sonrasında metinsel özelliklerle birleştirilmeye hazır bir matris hâline getirilmektedir. Böylece, satır sayısı gibi büyük ölçekli değişkenler ile karmaşıklık indeksleri gibi görece küçük ölçekli değişkenlerin, model parametrelerini orantısız şekilde etkilemesi engellenmiştir. Standartlaştırılmış metrikler, hem tek başına girdi vektörü olarak hem de metinsel gömülerle birleştirilmiş çok modlu temsiller içinde kullanılmıştır. Bu yaklaşım, farklı ölçek ve dağılıma sahip özelliklerin aynı öğrenme süreci içinde dengeli bir şekilde temsil edilmesine olanak tanımış ve özellikle sınıflar arası ayrım gücünün daha istikrarlı biçimde gözlemlenmesini sağlamıştır.

4.3.2. Metinsel Temsiller

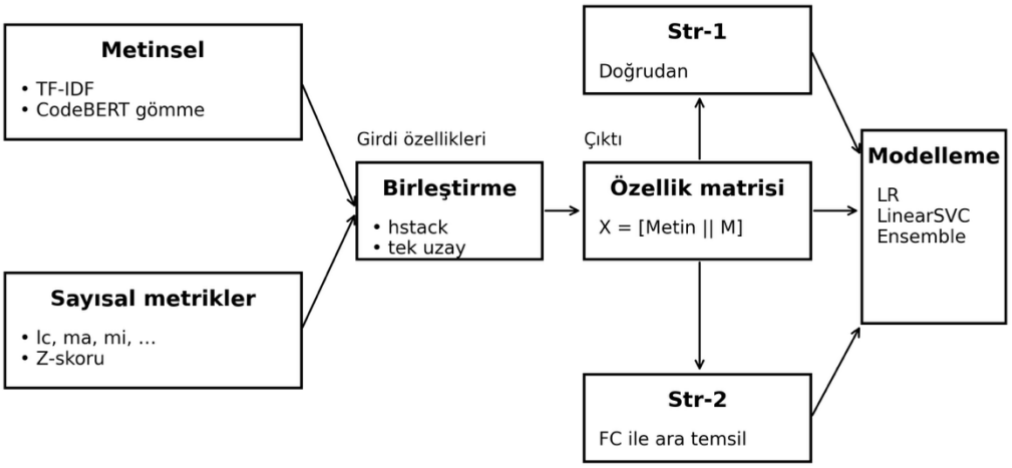
Metinsel temsiller için iki tamamlayıcı yaklaşım kullanılmıştır. Birinci yaklaşımda, kod ve yorum alanları birleştirilerek kelime ve karakter düzeyinde TF-IDF vektörleri çıkarılmıştır. Bu sayede, belirli anahtar kelimelerin, hata mesajlarının ve desenlerin frekans tabanlı öznitelik ağırlıkları modele girdi olarak verilmiştir. İkinci yaklaşımda ise CodeBERT tabanlı vektör temsilleri kullanılmıştır. Her kayıt için belirlenen metinsel alan, önceden eğitilmiş CodeBERT belirteçleyicisi ile tokenlara dönüştürülmüş ve modelin son gizli katmanından alınan temsiller örnek düzeyinde sabit boyutlu bir gömü vektörüne (embedding vector) dönüştürülmüştür.

CodeBERT tabanlı gömüler, kodun sözdizimsel yapısı ile geliştirici niyetini yansıtan isimlendirme ve kontrol-akışı ipuçlarını aynı anda taşıdığı için, hata şiddeti tahmini için zengin bir temsil alanı sunmaktadır. Bu çalışmada, hem yalnızca TF-IDF tabanlı metinsel özelliklerle eğitilen modeller hem de CodeBERT gömülerini kullanan modeller değerlendirilmiş. Ayrıca bu iki tipi birleştiren tekniklere de yer verilmiştir.

4.3.3. Çok Modlu Birleştirme

Bu kısımda farklı kaynaklardan elde edilen tüm özellikler tek bir birleşik vektörde toplanmaktadır. Çok modlu birleştirme aşamasında, metinsel temsiller ile sayısal metrikler tek bir özellik uzayında bir araya getirilmiştir. Bunun için iki temel strateji denenmiştir. İlk stratejide, CodeBERT vektörleri ile standartlaştırılmış metrikler yan yana eklenerek birleştirilmiştir. Bunlar klasik makine öğrenmesi sınıflandırıcılarına girdi olarak verilmiştir. İkinci stratejide ise gömü ve metrik bileşenleri üzerine ek tam bağlantılı katmanlar uygulanarak, lineer olmayan bir dönüşüm üzerinden ortak bir ara temsil öğrenilmiştir. SciPy kütüphanesinin seyrek matris fonksiyonları kullanılarak yatay biçimde birleştirilmektedir. Uygulamada hstack fonksiyonu ile TF-IDF, LLM gömmeleri ve sayısal özellikler tek bir özellik matrisi hâline getirilmektedir. Bu işlem sonucunda, her hata kaydı için hem metinsel içeriği hem de kodun yapısal özelliklerini taşıyan çoklu-modal bir özellik vektörü elde edilmektedir. Bu aşama, numpy ve scipy.sparse kütüphaneleri ile matris işlemleri kullanılarak, scikit-learn Pipeline yapıları içerisinde ardışık bir akış olarak gerçekleştirilmektedir.

Yorumlanabilirlik ve hesaplama maliyeti göz önüne alınarak, ağırlıklı olarak birinci strateji tercih edilmiştir. Yani, çok modlu temsiller, CodeBERT gömüleri ve sayısal metriklerin birleştirilmiş hali olarak ele alınmış ve bu temsiller üzerine lojistik regresyon, doğrusal destek vektör makineleri ve gradyan artırmalı ağaç tabanlı modeller gibi denetimli sınıflandırıcılar eğitilmiştir. Böylece, hem metrik tabanlı temel yöntemlerle karşılaştırma yapılabilmiş hem de büyük dil modeli gömülerinin ek katkısı açık biçimde gözlemlenebilmiştir. Birleştirilmiş özellik uzayı oluşturulduktan sonra, farklı makine öğrenmesi modelleri eğitim ve değerlendirme için kullanılmıştır. Bu çalışmada scikit-learn kütüphanesi ile Lojistik Regresyon, doğrusal destek vektör makineleri ve bu modelleri bir araya getiren topluluk (ensemble) yapıları denenmiştir. Metinsel ve sayısal özelliklerin tek bir uzayda birleştirilme süreci Şekil 4.3'de gösterilmektedir.



Şekil 2 Çok modlu birleştirme

4.3.4. Lojistik Regresyon

Lojistik regresyon, girdi özelliklerini doğrusal bir birleşimle modele alarak sınıflar için olasılık değerleri üreten denetimli bir sınıflandırma yöntemidir. Çok sınıflı durumda genellikle "one-vs-rest" veya softmax (multinomial) biçiminde çalıştırılır ve karar sınırlarını doğrusal bir hiper-düzlemle tanımlar. Düzenlileştirme (L1/L2) terimleri, yüksek boyutlu metinsel temsiller ve çok modlu özellik uzaylarında aşırı uyumu azaltmak ve genelleme performansını iyileştirmek için kritik rol oynar. Bu çalışmada LR, hem

yorumlanabilir bir temel model olması hem de seyrek/yoğun özellik vektörleri üzerinde hızlı ve kararlı sonuçlar üretmesi nedeniyle tercih edilmiştir.

4.3.5. Doğrusal Destek Vektör Makineleri (Linear SVM / LinearSVC)

Doğrusal SVM, sınıfları ayıran karar sınırını "marjı" en büyükleyecek şekilde öğrenen bir yöntemdir. Özellikle yüksek boyutlu ve seyrek temsillerde (ör. TF-IDF) güçlü bir taban çizgisi sağlamaktadır. Çünkü karar fonksiyonu doğrusal olup optimizasyonu verimlidir ve genellikle iyi genelleme gösterir. LinearSVC doğrudan sınıf etiketi üretir. Olasılık çıktısı üretmediği için, olasılık temelli analiz veya eşik ayarlaması gerektiğinde CalibratedClassifier gibi kalibrasyon yaklaşımlarıyla sınıf skorları olasılığa dönüştürülebilir. Bu sayede, sınıf dengesizliği veya "High/Critical" gibi kritik sınıflar için karar eşiğiyle oynanabilen daha esnek bir değerlendirme yapılabilir.

4.3.6. Topluluk (Ensemble) Yapıları

Topluluk yöntemleri, birden fazla sınıflandırıcının çıktısını birleştirerek tek bir modele göre daha dengeli ve kararlı tahmin üretmeyi hedefler. Temel fikir, farklı modellerin hata türlerinin kısmen farklı olmasından yararlanarak toplam hatayı azaltmaktır. Birleştirme çoğunluk oylaması (hard voting) ya da olasılıkların/puanların ağırlıklı ortalaması (soft voting) şeklinde yapılabilir. Ayrıca bazı durumlarda modellerin çıktıları ikinci bir "üst model" ile (stacking) birleştirilebilir. Bu tez kapsamında topluluk yaklaşımı, metinsel ve sayısal özelliklerin birlikte kullanıldığı durumda farklı sınıflandırıcıların güçlü yönlerini bir araya getirerek özellikle sınıf bazlı F1 ve makro F1 açısından daha tutarlı sonuçlar elde etmeyi amaçlamıştır.

Lojistik Regresyon, çok sınıflı şiddet tahmini için doğrusal bir karar sınırı sunmaktadır. Ceza terimi ve düzenleme katsayısı gibi hiperparametreler, ızgara arama ve çapraz doğrulama yöntemleri ile ayarlanmaktadır. LinearSVC ve CalibratedClassifier sınıfları kullanılarak SVM tabanlı modeller hem sınıf tahmini hem de kalibre edilmiş olasılık tahmini üretecek şekilde yapılandırılmıştır.

Ayrıca, birden fazla sınıflandırıcıyı bir arada kullanan topluluk modelleri ile, farklı algoritmaların güçlü yönlerinin birleştirilmesi hedeflenmiştir. Bu sayede, tek bir modele göre daha dengeli ve kararlı sonuçlar elde edilmiştir. Bu çalışma kapsamında, klasik tekli

modellere göre en yüksek doğruluk ve makro F1 puanını sağlayan yapı, bu topluluk yaklaşımı olmuştur.

Karşılaştırma amacıyla, etiketli veri kullanmadan çalışan sıfır-atış (zero-shot) büyük dil modeli deneyleri de gerçekleştirilmiştir. Bu senaryoda, modelden yalnızca dil bilgisine dayanarak bir hata kaydının hangi şiddet sınıfına daha yakın olduğu tahmin etmesi istenmiş ve bu sonuçlar, eğitilmiş modellerle elde edilen sonuçlarla karşılaştırılmıştır. Böylece, etiketli veri eksikliği durumunda kullanılabilecek bir alt sınır ortaya konulmuştur.

Tüm modeller, eğitim veri seti dosyasında yer alan eğitim verisiyle eğitilmiş ve test veri seti dosyasındaki bağımsız test verisi üzerinde değerlendirilmiştir. Değerlendirmede doğruluk, makro F1, ağırlıklandırılmış F1, sınıf bazlı başarı değerleri ve karışıklık matrisi gibi ölçütler kullanılmıştır. Ölçütler, çok sınıflı (K sınıf) hata şiddeti sınıflandırma problemi için verilmektedir. Özellikle Yüksek ve Kritik şiddet sınıflarındaki iyileşmeler ayrı olarak incelenmiş ve tartışılmıştır. Tanımlarda K, şiddet sınıfı sayısını, N ise test kümesindeki toplam örnek sayısını göstermektedir.

4.4. Performans Değerlendirme Kriterleri

Modellerin karşılaştırılabilir biçimde raporlanabilmesi için birden fazla değerlendirme ölçütü kullanılmıştır. Özellikle sınıf dengesizliğinin etkisini daha doğru yansıtmak amacıyla Macro-F1 ve Weighted-F1 ölçütleri birlikte ele alınmıştır.

Makro F1 (Macro-F1): Her sınıf için ayrı hesaplanan F1 değerlerinin aritmetik ortalamasıdır ve tüm sınıflara eşit ağırlık verir. Bu nedenle az görülen sınıflardaki performansı daha görünür kılar ve dengesiz sınıf dağılımı bulunan problemlerde güçlü bir özet ölçüt sunar.

Ağırlıklandırılmış F1 (Weighted-F1): Sınıf bazlı F1 değerlerinin, ilgili sınıfın test kümesindeki örnek sayısı ile ağırlıklandırılmasıyla elde edilen ortalamadır. Veri dağılımını dikkate aldığı için genel performansı özetlemede kullanışlıdır; ancak baskın sınıfların etkisi artabileceğinden azınlık sınıflarındaki hataları kısmen maskeleyebilir.

4.4.1. Karışıklık Matrisi (Confusion Matrix)

Karışıklık matrisi M, K×K boyutlu olup M(i,j) terimi gerçek sınıfı i olan örneklerin j sınıfı olarak tahmin edilme sayısını ifade eder. Diyagonal terimler M(i,i) doğru sınıflandırmaları, diyagonal dışı terimler ise hatalı sınıflandırmaları temsil eder.

4.4.2. Sınıf Desteği (Support)

Her bir sınıf için destek (n_i), ilgili sınıfın test kümesindeki örnek sayısıdır ve karışıklık matrisinin ilgili satır toplamı olarak denklem 4.1'de gösterildiği gibi hesaplanmıştır;

$$n_i = \sum_\{j=1..K\}\ M(i,j) \tag{4.1}$$

TP, FP, FN ve TN tanımları, çok sınıflı yapıda sınıf bazlı ölçütler "bir-sınıfa-karşı-diğerleri" (one-vs-rest) yaklaşımıyla hesaplanmıştır. Her sınıf i için aşağıdaki büyüklükler kullanılmıştır;

$$TP_i = M(i,i) \text{ (sınıf i doğru tahmin)} \tag{4.2}$$

$$FP_i = \sum M(r,i),\ r \in \{1..K\},\ r \neq i \text{ (i olarak tahmin edilip gerçekte i olmayanlar)} \tag{4.3}$$

$$FN_i = \sum M(i,c),\ c \in \{1..K\},\ c \neq i \text{ (gerçekte i olup i dışında tahmin edilenler)} \tag{4.4}$$

$$TN_i = N - TP_i - FP_i - FN_i \text{ (diğer tüm doğru kararlar)} \tag{4.5}$$

4.4.3. Doğruluk (Accuracy)

Doğruluk, tüm sınıflar üzerinde doğru tahmin edilen örneklerin toplam örnek sayısına oranıdır. Genel başarıyı özetler. Ancak sınıf dağılımı dengesiz olduğunda baskın sınıf lehine yüksek görünebilir. Bu nedenle doğruluk sonuçları, makro ortalamalı ölçütler ve sınıf bazlı sonuçlarla birlikte denklem 4.6'da görüldüğü gibi yorumlanmıştır.

$$\text{Accuracy} = (\sum_\{i=1..K\}\ TP_i) / N \tag{4.6}$$

4.4.4. Kesinlik ve Duyarlılık (Precision, Recall)

Her sınıf i için kesinlik, i olarak tahmin edilen örneklerin ne kadarının gerçekten i olduğunu, duyarlılık ise gerçekte i olan örneklerin ne kadarının doğru yakalandığını gösterir. Bu tezde sınıf bazlı Precision_i ve Recall_i değerleri, özellikle High ve Critical

sınıflarında yanlış alarm (FP) ve kaçırma (FN) davranışını göstermek amacıyla ayrıca raporlanmıştır.

$$\text{Precision}_i = TP_i / (TP_i + FP_i) \tag{4.7}$$

$$\text{Recall}_i = TP_i / (TP_i + FN_i) \tag{4.8}$$

4.4.5. F1 Ölçütü

F1, kesinlik ve duyarlılığın harmonik ortalamasıdır ve iki ölçüt arasındaki dengeyi tek bir değerle ifade eder. F1 ölçütü, özellikle dengesiz sınıflarda yalnızca doğruluğa kıyasla daha anlamlı bir performans özeti sunar.

$$F1_i = 2 \times (\text{Precision}_i \times \text{Recall}_i) / (\text{Precision}_i + \text{Recall}_i) \tag{4.9}$$

4.4.6. Makro ve Ağırlıklandırılmış Ortalama

Çok sınıflı değerlendirmede tek bir sayı ile özetleme yapmak için makro ve ağırlıklandırılmış ortalamalar kullanılmıştır. Makro ortalama her sınıfı eşit ağırlıkla dikkate alır. Ağırlıklandırılmış ortalama ise sınıf desteklerini (n_i) ağırlık olarak kullanır:

$$\text{Macro-F1} = (1/K) \times \sum_\{i=1..K\} F1_i \tag{4.10}$$

$$\text{Weighted-F1} = (1/N) \times \sum_\{i=1..K\} n_i \times F1_i \tag{4.11}$$

Sınıf dengesizliği nedeniyle modellerin karşılaştırılmasında birincil özet ölçüt olarak Macro-F1 tercih edilmiştir. Weighted-F1 ise veri dağılımını dikkate alan tamamlayıcı bir gösterge olarak sunulmuştur.

Sınıf bazlı başarı değerleri, her model için her sınıfta Precision_i, Recall_i ve $F1_i$ değerleri ve ilgili support (n_i) raporlanmıştır. Karışıklık matrisi ile birlikte değerlendirildiğinde, hangi şiddet seviyelerinin hangi sınıflarla karıştırıldığı açık biçimde görülebilmektedir.

4.5. Öğrenme Paradigmaları

Büyük dil modellerinin hata şiddeti tahminindeki performansı farklı öğrenme paradigmaları üzerinden değerlendirilmiştir. Analizlerde etiketli veri gereksinimi ve hesaplama maliyeti gibi temel kriterler dikkate alınmıştır. Bu doğrultuda sıfır örnekli (zero-shot), az örnekli (few-shot) ve tam ince ayar (fine-tuning) yöntemleri karşılaştırmalı olarak incelenmiştir. Ayrıca dil modellerinin yalnızca öznitelik çıkarıcı olarak kullanıldığı hibrit kurulumlar referans noktası olarak seçilmiştir. Böylece her bir yaklaşımın başarı oranı ve pratik uygulanabilirliği sistematik olarak ortaya konulmuştur.

4.5.1. Sıfır Örnekli (Zero-shot) Çıkarım

Sıfır örnekli çıkarım deneylerinde, doğal dil çıkarımına dayalı önceden eğitilmiş bir dil modeli (örneğin BART tabanlı bir model), kod parçacığı veya hata açıklaması ile şiddet etiketi arasında metinsel ilişki kurmak için kullanılmıştır. Bu senaryoda, model herhangi bir özel yeniden eğitim yapılmadan, yalnızca uygun istem cümleleri üzerinden tahmin üretmiştir. Örneğin, ilgili kayıt için, bu değişikliğin düşük, orta veya yüksek şiddette bir hataya yol açma olasılığını karşılaştıran metinsel kalıplar kullanılmıştır. Şekil 4.4'de sistemin genel mimarisi görülmektedir.

```
self.severity_labels = {
            0: "Low severity bug: Minor issue with minimal impact on functionality. Simple fix required. Low priority.",
            1: "Medium severity bug: Moderate issue affecting some functionality. Needs attention but not critical. Medium
priority.",
            2: "High severity bug: Major issue significantly impacting important features. Requires urgent fix. High priority.",
            3: "Critical severity bug: Severe issue causing system failures or security risks. Immediate fix mandatory. Critical
priority."
    }
```

Şekil 3.4. Örnek etiketlerin kullanımı ekran çıktısı

Bu yaklaşım, etiketli verinin kısıtlı olduğu veya yeni bir projeye hızlı uyum gereken durumlarda, büyük dil modellerinin önceden öğrenilmiş bilgi birikiminden faydalanma potansiyelini göstermektedir. Bununla birlikte, sıfır örnekli tahminler genellikle veri kümesine özel ince ayarlı modeller kadar yüksek performans sağlamadığından, bu tezde sıfır örnekli sonuçlar daha çok taban çizgisi (baseline) olarak raporlanmıştır.

4.5.2. Az Örnekli (Few-shot) Öğrenme

Az örnekli öğrenme senaryolarında, her sınıf için sınırlı sayıda (örneğin 5 ile 10 adet) etiketli örnek kullanılarak büyük dil modelleri kısmi olarak uyarlanmıştır. Bu amaçla, modelin taban katmanları sabit tutulmuş, yalnızca son gizli katmanlar ve sınıflandırma katmanı güncellenmiştir. Böylece hem hesaplama maliyeti düşürülmüş hem de küçük veri kümelerinde aşırı uyum riski azaltılmıştır. Deneysel sonuçlar, az örnekli ince ayarın, saf sıfır örnekli çıkarıma kıyasla özellikle Medium ve High sınıflarında anlamlı iyileşmeler sağladığını göstermektedir.

4.5.3. Tam İnce Ayar (Fine-tuning) Senaryoları

Tam ince ayar senaryolarında, büyük dil modelinin tüm parametreleri hata şiddeti tahmini görevine göre güncellenmiştir. Bu aşamada, AdamW benzeri ağırlık çürümesi içeren optimizasyon algoritmaları kullanılmış ve erken durdurma ile öğrenme oranı azaltma stratejileri uygulanmıştır. Guo ve diğ. (2024) tarafından vurgulandığı gibi, ince ayar sırasında katman dondurma desenleri, başlangıç ağırlıkları, öğrenme oranı ve ağırlık çürümesi gibi hiperparametreler sonuçlar üzerinde önemli etkiye sahiptir. Bu nedenle çalışma kapsamında, farklı ince ayar yapılandırmaları karşılaştırılmış ve en kararlı performansı sağlayan ayarlar seçilmiştir.

Ek olarak, bellek maliyetini azaltmak ve büyük modellerin daha az donanım ortamlarında da eğitilebilmesini sağlamak amacıyla, düşük dereceli uyarlama yaklaşımları (örneğin LoRA benzeri teknikler) değerlendirilmiştir. Bu sayede, modelin tüm ağırlıklarını güncellemeden yalnızca küçük bir alt uzayda güncelleme yaparak şiddet tahmin performansına katkı sağlanması hedeflenmiştir.

4.5.4. Özellik Tabanlı Klasik Makine Öğrenmesi Modelleri

Özellik tabanlı deneylerde, CodeBERT gömüleri ve sayısal metrikler önceden hesaplanmış sabit özellik vektörleri olarak ele alınmış ve bu vektörler üzerinde klasik makine öğrenmesi algoritmaları eğitilmiştir. Lojistik regresyon, doğrusal destek vektör makineleri, gradyan artırmalı sınıflandırıcılar ve bazı deneylerde rastgele orman ve XGBoost gibi ağaç tabanlı yöntemler kullanılmıştır. Bu kurulum, büyük dil modelinin yalnızca bir özellik çıkarıcı olarak kullanıldığı ve son kararın daha hafif modeller tarafından verildiği bir mimariyi temsil etmektedir.

Bu yaklaşımın avantajı, eğitim süresinin görece kısa olması ve hiperparametre aramasının daha sistematik biçimde gerçekleştirilebilmesidir. Ayrıca, şiddet sınıfları arasındaki karar sınırlarının incelenmesi ve özellik önemlerinin değerlendirilmesi açısından daha yorumlanabilir sonuçlar elde edilmektedir. Tezde, özellik tabanlı modeller aynı zamanda tam ince ayarlı büyük dil modeli tabanlı modeller için referans noktası olarak kullanılmıştır.

4.6. Değerlendirme ve Analiz

Model performansını değerlendirmek için, literatürde yaygın olarak kullanılan doğruluk, mikro ve makro ortalama F1 skorları ile ağırlıklı F1 skoru hesaplanmıştır. Şiddet odaklı bir problem olması nedeniyle, özellikle sınıf bazlı F1 skorlarına ve High sınıfının duyarlılık ve özgüllük değerlerine ayrı bir önem verilmiştir. Buna ek olarak, bazı deneylerde Matthews korelasyon katsayısı ve karışıklık matrisleri raporlanarak, sınıf dengesizliğinin etkileri ayrıntılı biçimde analiz edilmiştir.

Eğitim ve değerlendirme sırasında veri sızıntısını önlemek için, tüm önişleme ve ölçekleme adımları yalnızca eğitim verisi üzerinde öğrenilmiş ve aynı dönüşümler doğrulama ve test kümelerine uygulanmıştır. Eğitim veri seti ve test veri seti dosyalarının birbirinden bağımsız tutulmasına dikkat edilmiştir. Model seçimi ve hiperparametre ayarlamaları yalnızca eğitim ve doğrulama bölünmeleri kullanılarak gerçekleştirilmiştir. Elde edilen son çıktılar, eğitim sürecinde hiç kullanılmamış test kümesi üzerinde raporlanmıştır. Böylece, elde edilen sonuçların genellenebilirliği ve metodolojinin tekrarlanabilirliği güvence altına alınmıştır.

5. DENEYSEL DÜZENEK

Çalışma kapsamında gerçekleştirilen deneylerin çalıştırıldığı donanım ve yazılım ortamı, kullanılan kütüphaneler ve değerlendirme ölçütleri açıklanmaktadır. Tüm deneyler aynı deneysel kurulum altında yürütülmüş, böylece farklı modellerin ve özellik kümelerinin eşit koşullarda karşılaştırılması hedeflenmiştir. Bu doğrultuda deneylerde kullanılan işlemci, bellek, işletim sistemi, programlama dili sürümleri ve temel yazılım bağımlılıkları ayrıntılı biçimde sunulmuştur. Ayrıca veri ön işleme adımlarında yararlanılan araçlar, model eğitiminde kullanılan çerçeveler ve performans değerlendirmesinde esas alınan ölçütler de bu bölümde sistematik olarak ele alınmaktadır. Deneysel düzenek bölümünün amacı yalnızca kullanılan teknik altyapıyı tanıtmak değil, aynı zamanda gerçekleştirilen deneylerin tekrarlanabilirliğini ve elde edilen bulguların karşılaştırılabilirliğini güvence altına almaktır. Bu nedenle eğitim, doğrulama ve test süreçlerinde izlenen temel yaklaşım ile değerlendirme aşamasında dikkate alınan kriterler birlikte açıklanarak sonraki bölümlerde sunulacak sonuçların hangi koşullar altında elde edildiği açık bir çerçeve içerisinde ortaya konulmaktadır.

5.1. Çalışma Ortamı

Deneyler, Linux tabanlı bir işletim sistemi üzerinde çalışan, GPU hızlandırmalı bir sunucuda yürütülmüştür. Geliştirme ortamı olarak Python 3 sürümü ve Jupyter Notebook kullanılmıştır. Tüm kodlar tez kapsamında hazırlanan üç ana defterde (zero-shot.ipynb, few-shot.ipynb, fine-tuned.ipynb) organize edilmiştir.

Derin öğrenme ve makine öğrenmesi bileşenleri için PyTorch tabanlı bir altyapı tercih edilmiştir. Önceden eğitilmiş dil modellerinin yönetimi ve çağrılması için Transformers kütüphanesi, klasik makine öğrenmesi modelleri ve metrik hesaplamaları için scikit-learn, veri işleme ve analiz için ise pandas ve NumPy kullanılmıştır. Eğitilen modellerin kayıt, yeniden yükleme ve deney tekrarının sağlanması amacıyla rastgelelik kaynakları için sabit bir tohum (seed) değeri atanmıştır. Böylece farklı çalıştırmalarda tutarlı sonuçlar elde edilmesi amaçlanmıştır.

Deneyler boyunca kullanılan temel kütüphaneler PyTorch (derin öğrenme modellerinin tanımlanması ve eğitimi), Transformers (CodeBERT ve benzeri önceden eğitilmiş büyük dil modellerinin kullanımı), scikit-learn (lojistik regresyon, destek vektör makineleri,

kalibre edilmiş sınıflandırıcılar ve topluluk modellerinin eğitimi ve değerlendirilmesi) ile pandas ve NumPy (eğitim veri seti ve test veri seti veri kümelerinin yüklenmesi, birleştirilmesi ve ön işlenmesi) olarak özetlenebilir. Tüm deneyler, aynı yazılım sürümleri ve aynı veri kümeleri üzerinde çalıştırılmıştır. Bunun sonucunda zero-shot, few-shot ve tam ince ayar (fine-tuning) yaklaşımlarının karşılaştırılabilir olması sağlanmıştır.

5.2. Değerlendirme Ölçütleri

Test veri kümesinde sınıf dağılımı Low için 47, Medium için 294 ve High için 161 örnek olacak biçimde dengesizdir. Bu nedenle değerlendirme aşamasında yalnızca toplam doğruluk (accuracy) değeri yeterli görülmemiş, her bir sınıf için ayrı ayrı duyarlılık (recall), kesinlik (precision) ve F1-skoru hesaplanmıştır.

Zero-shot deneyleri için elde edilen ilk sonuçlar, genel doğruluğun yaklaşık olarak 0,675 ve makro ortalama F1-skorunun 0,557 seviyesinde olduğunu göstermiştir. Bu aşama, yalnızca önceden eğitilmiş dil modeline dayalı, etiket kullanılmayan bir başlangıç performansı olarak kabul edilmiştir. Sonraki adımda, TF-IDF tabanlı metinsel özellikler, statik kod metrikleri ve CodeBERT gömlemeleri ile beslenen klasik makine öğrenmesi modelleri ve topluluk yapıları değerlendirilmiştir. En iyi topluluk modeli, test kümesi üzerinde yaklaşık olarak 0,739 doğruluk ve 0,688 makro ortalama F1-skoru elde ederek zero-shot yaklaşıma göre anlamlı bir iyileşme sağlamıştır. Ayrıntılı nicel sonuçlar ve her bir modelin sınıf bazlı performansı, Bulgular ve Tartışma bölümünde ayrıntılı olarak ele alınmaktadır.

6. DENEYSEL SONUÇLAR VE TARTIŞMA

Bu bölümde tez kapsamında geliştirilen hata şiddeti tahmin modellerinin deneysel sonuçları sunulmakta ve elde edilen bulgular ayrıntılı olarak tartışılmaktadır. Öncelikle zero-shot büyük dil modeli yaklaşımlarının performansı özetlenmekte, ardından TF-IDF tabanlı metinsel özellikler, statik kod metrikleri ve CodeBERT gömlemelerini birleştiren denetimli makine öğrenmesi modellerinin sonuçları değerlendirilmektedir. Son olarak, en iyi modelin hata şiddeti seviyelerine göre davranışı ve araştırma hedefleri açısından genel bir yorum sunulmuştur.

6.1. Zero-Shot Model Sonuçları

Zero-shot deneyleri, herhangi bir ek eğitim veya ince ayar yapılmaksızın yalnızca büyük dil modelinin önceden öğrenilmiş bilgi ve doğal dil istemlerine (prompt) dayanarak hata şiddeti tahmini yapma kapasitesini ortaya koymaktadır. Bu kapsamda, üç sınıflı (Low, Medium, High) yapı üzerinde gerçekleştirilen zero-shot değerlendirmesinde, test kümesi için yaklaşık olarak %67,53 doğruluk ve 0,557 makro ortalama F1-skoru elde edilmiştir. Sınıf bazında bakıldığında, Low sınıfı için F1-skoru yaklaşık 0,39, Medium sınıfı için 0,77 ve High sınıfı için 0,51 civarındadır. Bu değerler, modelin özellikle Medium sınıfını ayırt etmede diğer sınıflara kıyasla daha başarılı olduğunu göstermektedir.

Karışıklık matrisi incelendiğinde, Low sınıfındaki 47 örnekten yalnızca 14'ünün doğru tahmin edildiği, 29 örneğin Medium ve 4 örneğin High olarak sınıflandırıldığı görülmektedir. Medium sınıfı için 294 örneğin 259'u doğru tahmin edilmiş, 6 örnek Low ve 29 örnek High olarak etiketlenmiştir. High sınıfında ise 161 örneğin 66'sı doğru tahmin edilirken, 4'ü Low ve 91'i Medium olarak sınıflandırılmıştır. Bu dağılım, zero-shot modelin sınıf dengesizliğinden etkilendiğini, özellikle Low ve High gibi görece az örnek içeren ve daha zor ayırt edilen sınıflarda hata yaptığını göstermektedir. Tablo 6.1'de zero-shot deneylerinde alının sonuçlar listelenmiştir. Bu bulgular, zero-shot yaklaşımının herhangi bir görev özel eğitim süreci olmaksızın belirli bir düzeyde sınıflandırma başarısı sağlayabildiğini, ancak özellikle az temsil edilen sınıflarda karar sınırlarının yeterince net oluşturulamadığını göstermiştir.

Tablo 6.1. Zero-shot deney sonuçları

Paradigma/Model	Uygulanan İyileştirme	Accuracy	Macro-F1	Weighted-F1	Not
Zero-shot / CodeT5	Doğrudan zero-shot çıkarım (referans kötü sonuç)	0,35	0,45	0,36	İlk denemeler
Zero-shot / Improved Severity Predictor	LLM temsilleri + sayısal metrikler ile hibrit özellik uzayı	0,65	0,55	0,63	Ön iyileştirme sürümü
Zero-shot / Improved Severity Predictor	Oversampling + hibrit ağırlıklandırma + hibrit özellik uzayı	0,675	0,557	0,651	Zero-shot tarafındaki en iyi sonuç

Zero-shot deneylerinde,kullanılan model de yapılan değişiklerin sonuçları Tablo 6.2'de listelenmiştir.

Tablo 6.2. Zero-shot ve diğer modellerin karşılaştırılması

Seviye	Model/Paradigma	Accuracy	Macro-F1	Açıklama
En kötü	Zero-shot / CodeT5	0,35	0,45	Doğrudan zero-shot (kötü temel)
Temel çizgi	Metric-Only / Gradient Boosting	0,538	0,482	Yalnızca statik kod metrikleri
Zayıf few-shot	Few-shot / LogReg_L1	0,541	0,514	L1 ile düşük performans
Zero-shot (erken)	Zero-shot / Improved Severity Predictor	0,65	0,55	Hibrit özellik uzayı (erken sürüm)
Zero-shot (iyileşt.)	Zero-shot (Oversampling+Hybrid) / Improved Severity Predictor	0,675	0,557	Sınıf dengeleme + hibrit ağırlıklandırma
Güçlü few-shot	Few-shot / Ensemble	0,739	0,688	Ensemble ile dengeli artış
En iyi 3-sınıf	Few-shot (Threshold Tuning) / Ensemble_Tuned	0,731	0,693	Eşik ayarlama ile Macro-F1 artışı
Fine-tune (not)	Fine-tuning / LLM CodeBERT	0,759	0,680	Fine-tuning

Özetle zero-shot yaklaşımı, etiketli veri gerektirmemesi, kurulum maliyetinin düşük olması ve kısa sürede başlangıç performansı sağlaması açısından avantajlıdır. Bununla

birlikte, elde edilen makro ortalama F1-skoru ve Low sınıfındaki düşük başarı, yazılım hata şiddeti tahmini gibi dengesiz ve kritik bir problemde zero-shot modelin tek başına yeterli olmadığını ortaya koymaktadır. Bu nedenle bu çalışma kapsamında, zero-shot sonuçlar bir başlangıç tabanı (baseline) olarak kabul edilmiş ve daha sonra denetimli, çok modlu (multi-modal) modeller ile iyileştirme yapılması hedeflenmiştir.

6.2. Few-Shot Model Sonuçları

Few-shot öğrenme, modelin bir görevi çok az sayıda etiketli örnek üzerinden "bağlam içinde" görerek hızlı biçimde uyum sağlamasını amaçlayan bir yaklaşımdır. Bu paradigmadaki temel fikir, modelin kapsamlı bir yeniden eğitim sürecine girmeden, örneklerden görev formatını ve sınıf ayrımını çıkarabilmesidir. Few-shot yaklaşımı, sınırlı örnek sayısına rağmen metinsel ve sayısal özelliklerden üretilen temsillerin sınıflandırma performansına etkisini gözlemlemek ve farklı sınıflandırıcıların bu koşullardaki davranışını karşılaştırmak için kullanılmıştır. Few-shot deneyleri kapsamında eğitilen modellerin test kümesi üzerindeki doğruluk ve makro F1 sonuçları Tablo 6.3'de sunulmaktadır.

Tablo 6.3. Few-shot Sonuçları

Kod/Aşama	Model & Paradigma Açıklaması	Accuracy	Macro-F1
F0	Few-shot (prompt tabanlı) → LogReg (L1): İstem içine az sayıda örnek eklenerek elde edilen temsil üzerinde lojistik regresyon, seyrek TF-IDF temsillerinde aşırı seçicilik nedeniyle zayıf genelleme.	0,541	0,514
F1	Few-shot → LogReg (L2): Aynı kurulumda L2 düzenlileştirme; özellik uzayında daha dengeli ceza ile sınıf ayrımı güçlendirilmiştir.	0,715	0,680
F2	Few-shot → Calibrated LinearSVC: Doğrusal SVM karar skorları olasılığa kalibre edilerek (ör. sigmoid/isotonic) sınıf kararları daha kararlı hâle getirilmiştir.	0,737	0,626
F3	Few-shot → Ensemble: Birden fazla bileşen modelin (ör. LogReg, SVM vb.) çıktıları birleştirilerek varyans azaltılmış ve daha dengeli sınıf başarımı elde edilmiştir.	0,739	0,688
F4 (İyileştirilmiş)	Few-shot (Threshold Tuning) → Ensemble_Tuned: Kalibrasyon/olasılık çıktıları üzerinde karar eşiği ayarlaması yapılarak özellikle kritik sınıflar için duyarlılık-kesinlik dengesi optimize edilmiştir.	0,731	0,693

Few-shot deneylerinde kullanılan bazı yöntemleri değiştirerek elde edilen iyileştirmeler Tablo 6.4'de sunulmaktadır.

Tablo 6.4. Adım bazlı iyileştirme etkisi.

Geçiş	Uygulanan yöntem	Accuracy	Macro-F1	Kısa yorum
F0 (Eski/Kötü) → F1	L1 → L2 düzenlileştirme	+0,17	+0,16	Aşırı seçicilik azaldı; genel ayrım gücü arttı.
F1 → F2	SVM + olasılık kalibrasyonu	+0,02	-0,05	Skorların olasılığa çevrilmesi ile kararlar daha kararlı.
F2 → F3	Model birleştirme (Ensemble)	+0,002	+0,06	Varyans azaldı; daha dengeli sınıf başarımı.
F3 → F4 (İyileştirilmiş)	Eşik ayarlama (Threshold tuning)	-0,008	+0,005	Hedef sınıflar için duyarlılık-kesinlik dengesi optimize edildi.

Few-shot modellerin Macro-F1 değerine göre sıralaması Tablo 6.5'de sunulmaktadır.

Tablo 6.5. Few-shot modellerin Macro-F1'e göre sıralaması.

Method	Macro-F1	Accuracy	Öne çıkan yön
Few-shot Ensemble_Tuned	0,693	0,731	Eşik ayarlama ile Macro-F1 maksimize edildi.
Few-shot Ensemble	0,688	0,739	Birleştirme ile dengeli performans.
Few-shot LogReg_L2	0,680	0,715	Düzenlileştirme ile güçlü temel çizgi.
Few-shot Calibrated LinearSVC	0,626	0,737	Kalibre olasılıklar ile karar stabilitesi.
Few-shot LogReg_L1	0,514	0,541	Eski/zayıf kurulum.

Model ailesi bazında karşılaştırma Tablo 6.6'de sunulmaktadır.

Tablo 6.6. Model ailesi bazında özet (LogReg / SVM / Ensemble).

Model ailesi	Kurulum	Accuracy	Macro-F1	Ne zaman tercih edilir?
Lojistik Regresyon	Few-shot LogReg_L1	0,541	0,514	Hızlı ön test, ancak dengesiz sınıflarda zayıf kalabilir.
Lojistik Regresyon	Few-shot LogReg_L2	0,715	0,680	Güçlü ve hızlı temel çizgi, seyrek TF-IDF için uygun.
SVM	Few-shot Calibrated LinearSVC	0,737	0,626	Skor kalibrasyonu gerekli ise, karar eşiği analizleri için uygun.
Ensemble	Few-shot Ensemble	0,739	0,688	Daha dengeli sınıf başarımı, tek model oynaklığını azaltır.
Ensemble	Few-shot Ensemble_Tuned	0,731	0,693	Hedef sınıflara göre eşik ayarı yapıldığında en iyi Macro-F1.

6.3. Denetimli Makine Öğrenmesi Modellerinin Sonuçları

Denetimli deneylerde, eğitim veri seti ve test veri seti veri kümeleri kullanılarak TF-IDF tabanlı metinsel özellikler, statik kod metrikleri ve CodeBERT gömlemeleri birleştirilmiş ve bu hibrit özellik uzayı üzerinde çeşitli sınıflandırıcılar eğitilmiştir. Bu bağlamda, L2 cezalık Lojistik Regresyon (LogReg_L2), L1 cezalık Lojistik Regresyon (LogReg_L1), kalibre edilmiş destek vektör sınıflandırıcısı (Calibrated_SVC) ve bir topluluk (ensemble) modeli değerlendirilmiştir. Elde edilen sonuçlara göre, LogReg_L2 modeli test kümesi üzerinde yaklaşık 0,715 doğruluk ve 0,680 makro ortalama F1-skoru üretmiş, LogReg_L1 modeli ise 0,542 doğruluk ve 0,514 makro F1-skoru ile daha düşük bir performans sergilemiştir.

Calibrated_SVC modeli, yaklaşık 0,737 doğruluk ile yüksek bir genel başarı elde etmiş, ancak makro ortalama F1-skoru 0,627 seviyesinde kalmıştır. Bu durum, sınıf dengesizliği altında yalnızca doğruluk değerine bakmanın yeterli olmadığını, zayıf temsil edilen sınıflardaki performansın da ayrı ayrı incelenmesi gerektiğini göstermektedir. Topluluk modeli (Ensemble), farklı taban sınıflandırıcıların çıktılarını birleştirerek test kümesi üzerinde yaklaşık 0,739 doğruluk ve 0,688 makro ortalama F1-skoru ile en iyi sonucu vermiştir. Böylece, hibrit özellik kullanımı ve topluluk yaklaşımı bir araya geldiğinde hem doğruluk hem de makro F1 açısından zero-shot modele göre belirgin bir iyileşme sağlandığı görülmektedir.

LogReg_L2 ve Calibrated_SVC gibi modellerin, ağırlıklı F1-skoru ve karışıklık matrisleri incelendiğinde Medium sınıfında yüksek, Low ve High sınıflarında ise daha sınırlı bir başarıya sahip olduğu gözlenmiştir. Ensemble modeli ise, hibrit özellikler ve birden fazla sınıflandırıcının oy birliği yaklaşımı sayesinde, özellikle High sınıfında daha dengeli bir performans profili sergilemiştir. Bu bulgu, metinsel içerik (örneğin commit mesajları ve kod açıklamaları) ile statik kod metriklerinin birlikte kullanılmasının, kritik şiddet seviyelerini ayırt etme açısından tek modlu modellere göre avantaj sağladığını desteklemektedir.

6.4. Eşik Ayarlaması ve Topluluk Modeli Performansı

Çalışma kapsamında, topluluk modelinin sınıf olasılıkları üzerinde eşik ayarlaması yapılarak dengesiz sınıf dağılımına karşı daha duyarlı bir yapı elde edilmeye çalışılmıştır.

Bu amaçla, her sınıf için farklı eşik çarpanları denenmiş ve en iyi sonuç veren kombinasyon yaklaşık olarak [1,2; 1,0; 1,3] olarak bulunmuştur. Bu eşik ayarlaması sonrasında, Ensemble_Tuned modeli test kümesi üzerinde yaklaşık 0,731 doğruluk ve 0,693 makro ortalama F1-skoru üretmiştir.

Dikkat çekici nokta, eşik ayarlaması sonrasında toplam doğruluk değeri en iyi topluluk modeline kıyasla bir miktar düşmüş olmasına rağmen, makro ortalama F1-skorunun artmasıdır. Bu durum, özellikle veri setinde az temsil edilen Low ve High sınıflarında hatalı sınıflandırmaların kısmen azaltılması ve bu sınıfların daha dengeli bir şekilde ele alınması ile açıklanabilir. Başka bir deyişle, eşik ayarlaması, modelin dengesizliğe duyarlılığını artırarak her sınıfa daha yakın ağırlıkta bir başarı elde etmesine katkı sağlamıştır.

Pratik açıdan bakıldığında, kurumsal bir yazılım projesinde kritik öneme sahip olan High sınıfının daha doğru tahmin edilmesi, belli ölçüde genel doğruluktan fedakârlık edilmesini kabul edilebilir kılabilir. Bu nedenle, makro ortalama F1-skorundaki artış ve özellikle yüksek öncelikli hata sınıflarındaki iyileşme, eşik ayarlamasının operasyonel açıdan anlamlı bir kazanım sunduğunu göstermektedir.

6.5. Zero-Shot ve Denetimli Yaklaşımların Karşılaştırılması

Zero-shot yaklaşım ile denetimli hibrit modeller arasındaki karşılaştırma, büyük dil modellerinin etiketli veri olmadan dahi belirli bir performans seviyesine ulaşabildiğini, ancak dengesiz çok sınıflı bir problemde bunun sınırlı kaldığını göstermektedir. Zero-shot model yaklaşık 0,557 makro F1-skoru sağlarken, hibrit özelliklere dayanan en iyi topluluk modeli 0,688 makro F1-skoru ile bu değeri anlamlı biçimde aşmıştır. Bu fark hem metinsel hem de sayısal özelliklerin birlikte kullanıldığı, denetimli ve kalibre edilmiş bir öğrenme sürecinin, yalnızca önceden eğitilmiş bir modelin sezgisel çıkışlarına kıyasla daha güvenilir ve dengeli sonuçlar ürettiğini göstermektedir.

Bununla birlikte, zero-shot yöntemin etiketli veri gerektirmemesi nedeniyle veri toplamanın zor veya maliyetli olduğu projelerde hızlı bir başlangıç çözümü sunabileceği unutulmamalıdır. Tez kapsamında geliştirilen yaklaşım, zero-shot sonuçlarını bir taban çizgisi olarak kullanmakta ve ardından denetimli hibrit modellerle bu tabanın sistematik biçimde nasıl iyileştirilebileceğini göstermektedir. Böylece, büyük dil modellerinin hem

sıfır atış (zero-shot) hem de zenginleştirilmiş, çok modlu denetimli senaryolarda nasıl değerlendirilebileceğine dair bütüncül bir çerçeve sunulmuştur.

6.6. Hata Şiddeti Seviyelerine Göre Tartışma

Üç sınıflı yapı (Low, Medium, High) üzerinden yapılan analizler, Medium sınıfının gerek zero-shot gerekse denetimli modeller için nispeten daha kolay ayrıştırılabildiğini göstermektedir. Bunun başlıca nedenleri arasında, veri kümesinde Medium örneklerinin sayısının diğer sınıflara göre yüksek olması ve bu sınıfa ait commit mesajları ile kod değişikliklerinin daha tipik kalıplar içermesi sayılabilir. Buna karşılık, Low sınıfında yer alan örnek sayısının az olması ve etiketlemelerin görece daha öznel olabilmesi, modellerin bu sınıfı doğru tahmin etmesini zorlaştırmaktadır.

High sınıfı, kritik hataların da dâhil olduğu üst seviye şiddet grubunu temsil etmekte olup, sınıf dengesizliğinden en çok etkilenen gruplardan biridir. Zero-shot modelde High sınıfına ait birçok örneğin Medium sınıfına kaydığı, denetimli hibrit modellerde ise bu kaymanın kısmen azaldığı görülmektedir. Özellikle topluluk modeli ve eşik ayarlaması sonrasında High sınıfındaki F1-skorunun iyileşmesi, kod metrikleri ile metinsel özelliklerin birlikte kullanılmasının, yüksek şiddetli hataları önceliklendirirken daha isabetli tahminler yapılmasına katkı sağladığını göstermektedir.

Orijinal veri kümesinde yer alan Critical sınıfının High sınıfı ile birleştirilmesi hem sınıf dengesizliğini azaltmaya hem de modelin öğrenmesi gereken karar sınırlarını sadeleştirmeye yönelik bir tasarım tercihidir. Bu dönüşüm sonrasında elde edilen sonuçlar, üç sınıflı yapının hem zero-shot hem de denetimli modeller için daha kararlı bir değerlendirme zemini sunduğunu göstermektedir. Gelecek çalışmalarda, bu iki sınıfın yeniden ayrıştırılması ve özellikle en kritik hataların ayrı bir kategori olarak daha hassas yöntemlerle modellenmesi mümkün olabilir.

6.7. Araştırma Hedefleri Açısından Değerlendirme

Tezde belirlenen araştırma hedefleri dikkate alındığında, elde edilen deneysel sonuçlar genel olarak hedeflerle uyumlu bir tablo çizmektedir. Öncelikle, hata tahmini probleminin yalnızca “hata var/yok” ikili sınıflandırması yerine şiddet temelli çok sınıflı bir problem olarak ele alınması sağlanmış ve Low, Medium, High seviyeleri için ayrı ayrı performans

ölçümleri raporlanmıştır. Böylece, şiddet odaklı tahminin yazılım bakım süreçlerine nasıl entegre edilebileceğine ilişkin nicel bir temel oluşturulmuştur.

İkinci olarak, büyük dil modellerinden elde edilen metinsel gömlemelerin, TF-IDF temsilleri ve statik kod metrikleri ile birleştirildiği hibrit bir mimarinin, yalnızca tek modlu modellere göre daha yüksek makro F1-skoru sağladığı gösterilmiştir. En iyi topluluk modeli ile zero-shot yaklaşım arasındaki fark, büyük dil modellerinin kod ve commit mesajlarından öğrendikleri anlamsal temsillerin, denetimli bir çerçevede yeniden kalibre edildiğinde hata şiddeti tahmini için önemli bir potansiyel taşıdığını göstermektedir.

Son olarak, eşik ayarlaması ve sınıf dengesizliğine duyarlı değerlendirme ölçütlerinin (makro F1, ağırlıklı F1, karışıklık matrisi) kullanılması, model performansının yalnızca ortalama doğruluk açısından değil, her bir şiddet seviyesi açısından da ayrıntılı olarak anlaşılmasına katkı sağlamıştır. Bu sayede, önerilen yaklaşımın pratik yazılım projelerinde, özellikle de yüksek riskli hataların önceliklendirilmesi gereken durumlarda, nasıl konumlandırılabileceğine dair somut çıkarımlar elde edilmiştir. Bulgular ve tartışmalar, bir sonraki bölümde sunulan sonuçlar ve öneriler ile birlikte tez kapsamındaki katkıların bütüncül bir değerlendirmesini tamamlamaktadır.

7. SONUÇLAR VE ÖNERİLER

Çalışmanın kapsamı doğrultusunda gerçekleştirilen deneysel değerlendirmelerin sonuçları bütüncül biçimde sunulmakta ve elde edilen bulgular yorumlanmaktadır. Farklı öğrenme paradigmaları altında geliştirilen yaklaşımların test kümesi üzerindeki performansları karşılaştırılarak, şiddet sınıflandırma probleminde metinsel temsiller, sayısal metrikler ve hibrit stratejilerin katkısı tartışılmaktadır. Ayrıca bulgular ışığında, yöntemin uygulanabilirliği ve gelecekte yapılabilecek iyileştirmelere yönelik öneriler paylaşılmaktadır.

7.1. Genel Değerlendirme

Başlangıçta dört seviyeli şiddet yapısı (Low, Medium, High, Critical) ele alınmış, ancak veri kümesindeki aşırı dengesizlik ve sınırlı örnek sayısı nedeniyle Critical sınıfının High sınıfı ile birleştirilmesi sonucunda üç sınıflı bir yapı benimsenmiştir. Bu dönüşüm hem model kararlılığını artırmış hem de özellikle makro F1 skorları üzerinde iyileşme sağlamıştır. Çalışmada, gerçek dünyadan elde edilen eğitim veri seti ve test veri seti veri kümeleri kullanılmış, her bir kayıt için metinsel alanlar (kod, yorum içeren kod, yorumdan arındırılmış kod) ile sayısal metrikler (örneğin satır sayısı ve karmaşıklık ölçüleri) bir araya getirilmiştir. Metin tarafında, TF-IDF temsilleri ile büyük dil modeli tabanlı gömlemeler birlikte değerlendirilmiş, sayısal tarafta ise ölçeklenmiş kod metrikleri kullanılmıştır. Bu sayede hem kodun anlamsal yapısı hem de yapısal özellikleri aynı sınıflandırma hattı içinde temsil edilmiştir.

7.2. Model Performansı ve Çıkarımlar

Deney sonuçları, yalnızca metinsel özniteliklere dayanan zero-shot yaklaşımları ile çok modlu hibrit topluluk (ensemble) modelleri arasında belirgin bir performans farkı olduğunu ortaya koymaktadır. Model eğitimi gerektirmeyen zero-shot senaryosunda başarı sınırlı kalırken anlamsal özelliklerin sayısal kod metrikleriyle harmanlandığı topluluk modelleri, çok daha kararlı ve dengeli bir performans sergilemiştir. Bu durum, hata şiddetinin kestirilmesinde hibrit temsil ve topluluk modelleme stratejisinin etkinliğini doğrular niteliktedir. Farklı öğrenme paradigmalarına ait karşılaştırmalı test sonuçları, doğruluk, makro ortalama F1 puanı ve değişken sınıf sayılarına (3 ve 4 sınıflı kurulumlar) göre Tablo 7.1'de özetlenmiştir.

Tablo 7.1. Test kümesinde modellerin karşılaştırmalı performansı

Paradigma	Model	Sınıf Sayısı	Accuracy	Macro-F1
Few-shot	LogReg_L2	3	0,715	0,680
Few-shot	LogReg_L1	3	0,541	0,514
Few-shot	Calibrated_SVC	3	0,737	0,626
Few-shot	Ensemble	3	0,739	0,688
Few-shot (Threshold Tuning)	Ensemble_Tuned	3	0,731	0,693
Zero-shot (Oversampling + Hybrid)	Improved Severity Predictor	3	0,675	0,557
Fine-tuning	LLM CodeBERT	4	0,759	0,680
Baseline (Sayısal Metrik)	Gradient Boosting (Metric-Only)	3	0,538	0,482
Zero-shot	Improved Severity Predictor	3	0,65	0,55
Fine-tuning	CodeT5	4	0,75	0,68
Zero-shot	CodeT5	4	0,35	0,45
Few-shot	CodeT5	4	0,39	0,52

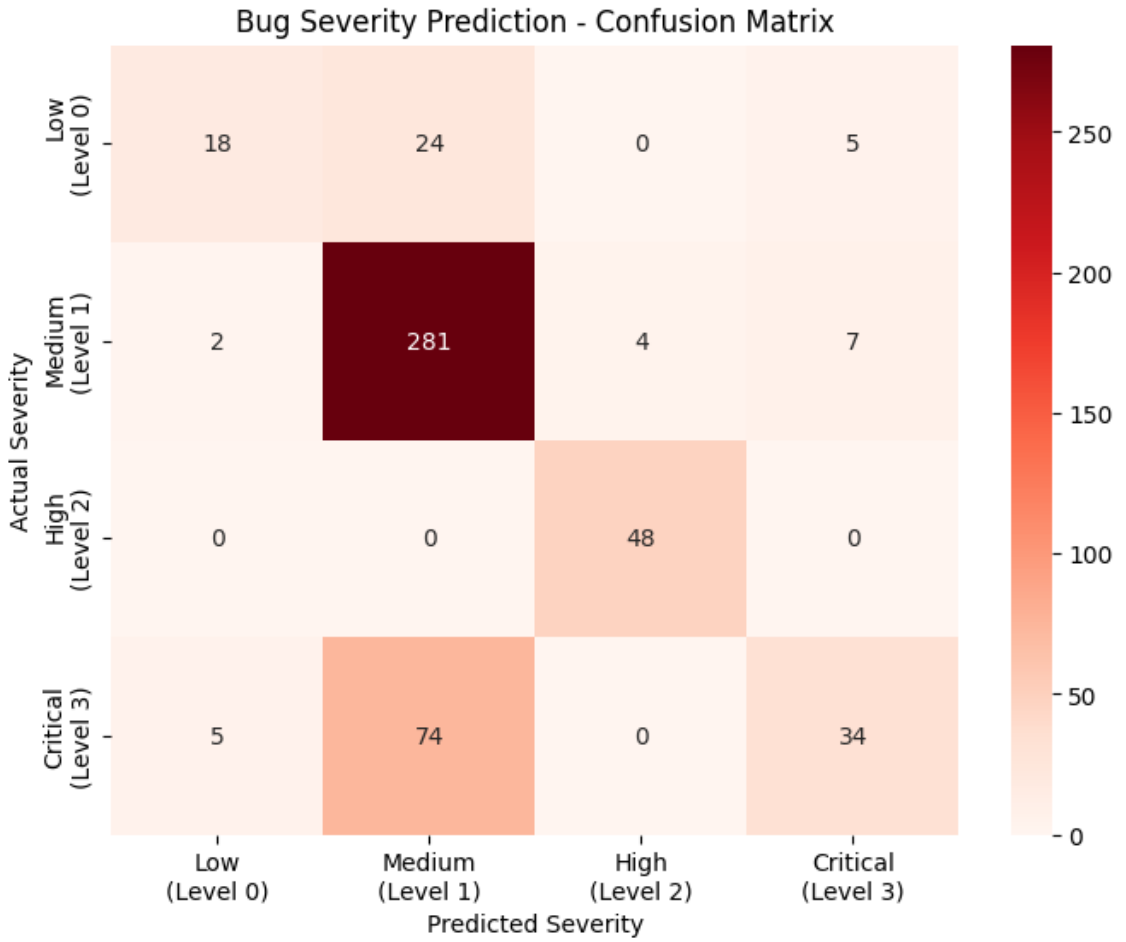
Tablo 7.1'de, üç sınıflı kurulumda few-shot yöntemleri arasında en dengeli sonuçların Ensemble ve Ensemble_Tuned yaklaşımlarıyla elde edildiği görülmektedir. Ensemble modeli doğruluk açısından 0,7390 değerine ulaşırken, eşik ayarlaması (threshold tuning) yapılan Ensemble_Tuned yaklaşımı makro F1 değerini 0,6933 seviyesine taşımıştır. Buna karşın, LogReg_L1 yaklaşımı hem doğruluk hem de makro F1 bakımından belirgin biçimde geride kalmıştır. Codebert ile karşılaştırmak için kullandığım CodeT5 modelinin sonuçları da tabloda gösterilmiştir.

Improved Severity Predictor, zero-shot paradigma kapsamında geliştirilen ve metin tabanlı LLM çıktılarının sayısal yazılım metrikleriyle birlikte ele alındığı hibrit bir yaklaşımdır. Bu modelde, CodeBERT benzeri ön-eğitimli dil modellerinden elde edilen anlamsal temsil vektörleri, oversampling ile dengelenmiş veri üzerinde değerlendirilmiş ve sınıf tahmini herhangi bir görev-özgü ince ayar (fine-tuning) yapılmadan gerçekleştirilmiştir. Elde edilen sonuçlar, özellikle orta şiddet sınıfında yüksek duyarlılık sağlandığını, ancak düşük ve yüksek şiddet sınıflarında sınıf ayrımının yeterince güçlü olmadığını göstermektedir. Bu durum, zero-shot yaklaşımların etiketli veri gereksinimi olmadan hızlı bir başlangıç sunduğunu, ancak daha dengeli ve güvenilir sonuçlar için

few-shot veya fine-tuning temelli yöntemlerle desteklenmesi gerektiğini ortaya koymaktadır.

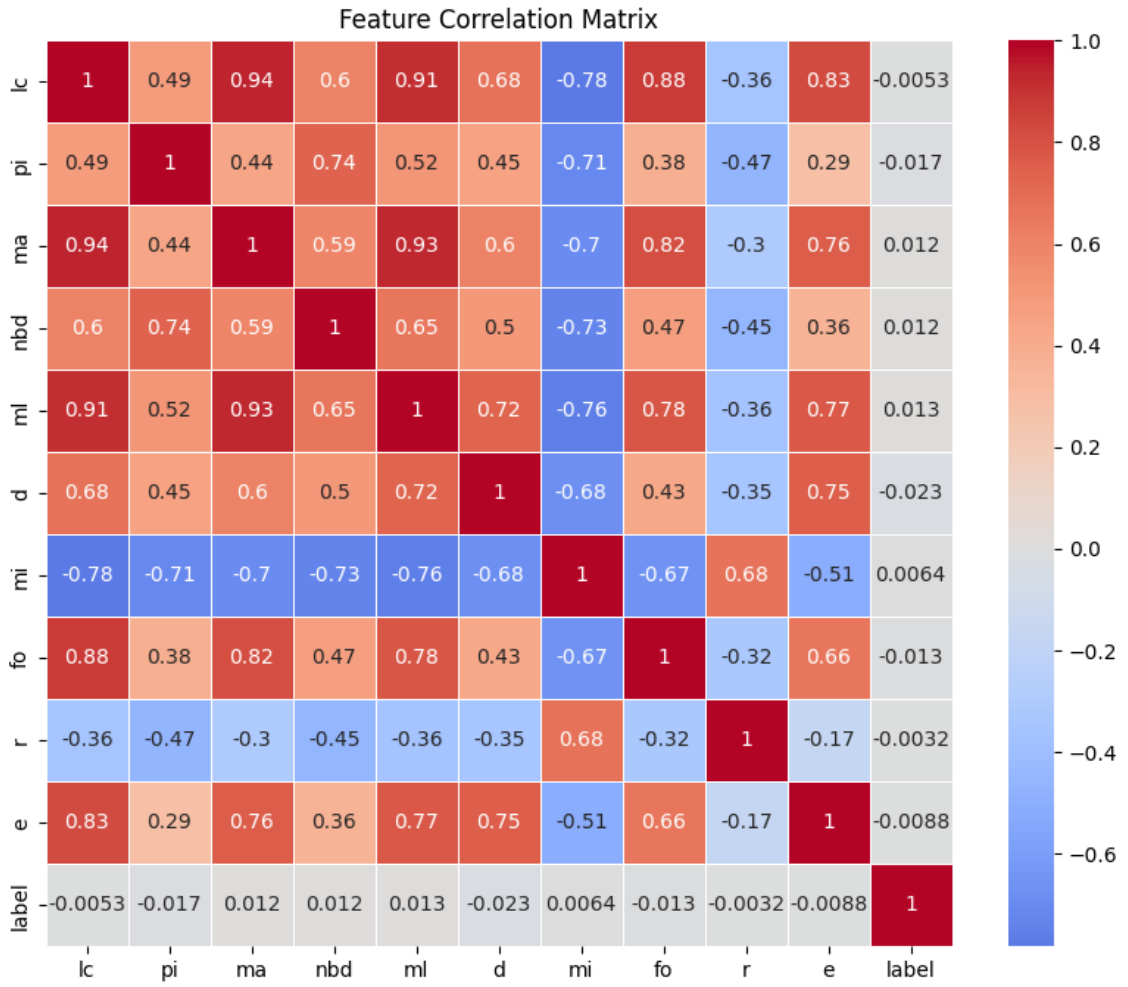
Zero-shot, oversampling ve hibrit ağırlıklandırma içeren yapı belirli bir iyileştirme hedeflese de test sonuçları (Accuracy=0.675, Macro-F1=0.557) few-shot temelli modellere kıyasla daha düşük kalmıştır. Bu durum, yalnızca istem tasarımı ve sınırlı bağlamsal yönlendirme ile elde edilen yaklaşımın, hata şiddeti gibi sınıflar arası sınırları daha hassas olan bir problemde tek başına yeterli düzeye ulaşmakta zorlanabildiğini göstermektedir. Özellikle sınıf dağılımındaki dengesizlik ve teknik hata kayıtlarının taşıdığı karmaşık anlamsal yapı, zero-shot yaklaşımın genelleme gücünü sınırlandırmış olabilir. Buna karşılık fine-tuning ile eğitilen LLM CodeBERT modeli dört sınıflı kurulumda 0.759 doğruluğa ve 0.680 makro F1 değerine ulaşarak daha güçlü bir performans sergilemiştir. Bu bulgu, alan uyumlu önceden eğitilmiş modellerin görev özelinde yeniden eğitildiğinde daha ayırt edici temsiller üretebildiğine işaret etmektedir. Bununla birlikte sınıf sayısının farklı olması nedeniyle bu sonucun üç sınıflı kurulumla doğrudan ve birebir aynı düzlemde değerlendirilmesi metodolojik açıdan uygun değildir; dolayısıyla yorum yapılırken veri yapısı, sınıf birleşimleri ve değerlendirme çerçevesi birlikte dikkate alınmalıdır.

Yazılım hata kestirimi literatüründe klasik bir makine öğrenmesi yaklaşımı olarak tercih edilen Gradient Boosting (Metric-Only) modeli, bu çalışmada temel çizgi (baseline) performansını belirlemek amacıyla kurgulanmıştır. Bu model, yalnızca statik kod analizinden elde edilen nicel metrikleri (satır sayısı, McCabe karmaşıklığı, Halstead parametreleri vb.) öznitelik uzayı olarak kullanmakta, kodun metinsel ve anlamsal (semantic) içeriğini ise analiz dışı bırakmaktadır. Yapılan deneylerde bu modelin, özellikle sınıflar arası dengesizliğin yüksek olduğu veri setlerinde, karmaşık hata örüntülerini yakalamakta yetersiz kaldığı gözlemlenmiştir. Sayısal metriklerin tek başına sunduğu bilgi kapasitesinin, yazılımın mantıksal akışındaki derin anlamsal özelliklerini temsil etmede kısıtlı kalması, modelin Macro-F1 skorunun LLM tabanlı yaklaşımların gerisinde kalmasına neden olmuştur. Elde edilen düşük performans çıktıları, hata tahmininde sadece yapısal metriklerin değil, kodun bağlamsal bilgisini işleyebilen dil modellerinin entegrasyonunun gerekliliğini göstermektedir. Şekil 7.2'de fine tune çalışma çıktısı gösterilmiştir.



Şekil 4 İnce ayarlı Codebert ile elde edilen karmaşıklık matrisi

Zero-shot deneyleri, etiketli veri miktarının sınırlı olduğu durumlarda büyük dil modellerinin anlamlı bir başlangıç performansı sağlayabildiğini göstermektedir. Hiçbir görev özelinde ince ayar yapılmadan elde edilen sonuçlar, klasik metrik temelli bazı yaklaşımlara yakın değerler üretmiş, ancak özellikle düşük örnek sayısına sahip sınıflarda hassasiyet ve geri çağırma değerleri sınırlı kalmıştır. Bu bulgu, zero-shot yaklaşımların erken dönemde kaba bir ön filtreleme ya da önceliklendirme aracı olarak kullanılabileceğini, ancak üretim ortamında tek başına yeterli olmadığını ortaya koymaktadır. Şekil 7.3'de codebert çalışma çıktısı özellikler ile birlite gösterilmiştir. Bu nedenle zero-shot yaklaşımı, çalışmada daha gelişmiş öğrenme stratejileriyle elde edilen sonuçların değerlendirilmesinde temel bir karşılaştırma düzeyi olarak ele alınmıştır.



Şekil 5 Etiket özellik korelasyon matris gösterimi

Şekil 7.3'de görüldüğü üzere, metrikler arasındaki doğrusal ilişkiyi gösteren bir korelasyon matrisi oluşturulmuştur. Özellikle lc–ma (0,94), ma–ml (0,93) ve lc–ml (0,91) gibi çiftlerde oldukça yüksek pozitif korelasyon var. Yani bu metrikler benzer bilgiyi taşıyor olabilir ve birlikte kullanıldıklarında çoklu doğrusal bağlantı (multicollinearity) riski doğurabilir. Buna karşılık mi metriği birçok metrikle güçlü negatif ilişkiye sahip olduğu görülüyor. Örneğin lc–mi -0,78, ml–mi -0,76 incelersek burada mi'nin "diğerleri arttıkça azalan" bir davranış sergilediğini düşündürüyor. Ayrıca label sütunu ve satırı neredeyse tüm metriklerle sıfıra yakın korelasyon gösteriyor. Bu da etiketin metriklerle basit doğrusal ilişkiyle açıklanmasının zor olduğunu ve daha karmaşık çok boyutlu bir modellemenin gerekebileceğini işaret ediyor. Tablo 7.2'de ve Tablo 7.3'de zero shot sonuçlarında da benzer sonuçlar görülebilinmektedir. Weighted F1 = 0,65 olarak oluşturulmuştur.

Tablo 7.2. Sınıf bazlı değerlendirme ölçütleri.

Sınıf / Özet	Precision	Recall	F1-score	Support
Low	0,5833	0,2979	0,394	47
Medium	0,6834	0,8810	0,769	294
High	0,6667	0,4099	0,507	161
Accuracy	0,675	0,675	0,675	502
Macro avg	0,6445	0,5296	0,557	502
Weighted avg	0,6687	0,6753	0,650	502

Tablo 7.3. Karışıklık matrisi (Satır: Gerçek sınıf, Sütun: Tahmin edilen sınıf).

Gerçek \ Tahmin	Low	Medium	High
Low	14	29	4
Medium	6	259	29
High	4	91	66

Few-shot senaryolarda, sınırlı sayıda etiketli örnek ile gerçekleştirilen ince ayar ve sonrasında uygulanan eşik ayarlama denemeleri, özellikle orta ve yüksek şiddet sınıflarında performansı iyileştirmiştir. Eşik çarpanlarının sistematik olarak tarandığı deneylerde, bazı sınıflar için karar sınırlarının yeniden ölçeklendirilmesi sonucunda yaklaşık 0,731 doğruluk ve 0,693 makro F1 skoru elde edilmiştir. Bu sonuçlar, sınıf dengesizliğinin yaygın olduğu hata şiddeti problemlerinde, yalnızca model mimarisi kadar karar eşiklerinin de kritik rol oynadığını göstermektedir.

Hibrit temsil yaklaşımı, yani büyük dil modeli tabanlı metinsel gömlemeler ile statik kod metriklerinin bir arada kullanılması, hem zero-shot hem de denetimli öğrenme deneylerine kıyasla daha dengeli bir performans profili ortaya koymuştur. Özellikle üç sınıflı yapı içinde yüksek şiddet düzeyinin (High) doğru tahmin edilmesi, önerilen yöntemin pratik değerini artırmaktadır. Elde edilen sonuçlar, yazılım ekiplerinin sınırlı test ve bakım kaynaklarını gerçekten yüksek risk taşıyan hatalara yönlendirebilmesi için, şiddet duyarlı tahmin modellerinin güçlü bir destek aracı olabileceğini göstermektedir.

7.3. Öneriler ve Gelecek Çalışmalar

Bu çalışmanın bulguları hem akademik araştırmalar hem de endüstriyel uygulamalar için bir dizi öneri sunmaktadır. Öncelikle, hata şiddeti tahmini problemlerinde dört sınıflı yapının doğrudan kullanılmasından ziyade, veri dağılımı ve örnek sayıları göz önünde

bulundurularak sınıf birleştirme ya da yeniden tanımlama stratejilerinin değerlendirilmesi önerilmektedir. Özellikle çok nadir görülen Critical sınıfı için, ayrı bir sınıflandırma aşaması ya da ikili risk modeli tasarlamak, ileride ele alınabilecek alternatiflerden biri olarak öne çıkmaktadır.

Gelecek çalışmalar için bir diğer önemli yön, commit düzeyinde anlık tahmin (Just-In-Time) senaryolarının sisteme entegrasyonudur. Bu tezde kullanılan veri kümeleri, ağırlıklı olarak kod ve metrik odaklıdır. Commit mesajları, hata raporları ya da kod inceleme yorumları gibi ek doğal dil sinyallerinin de modele dahil edilmesi, özellikle inceleme sürecinin erken aşamalarında geliştiricilere daha zengin açıklamalar sunabilir. Bu bağlamda, iki modlu ya da çok modlu temsil öğrenen modellerin, şiddet tahmini performansı üzerindeki etkisi ayrıntılı olarak incelenmelidir.

Ayrıca, daha büyük ve çeşitli veri kümeleri üzerinden çapraz proje değerlendirmeleri yapılması, modelin genellenebilirliğini test etmek açısından önemlidir. Farklı dil ve mimarilerdeki projeler üzerinde, aynı hibrit çerçevenin ne ölçüde başarılı olduğu, ileride araştırılması gereken temel sorulardan biridir. Buna ek olarak, daha gelişmiş büyük dil modelleri (örneğin, daha yeni nesil kod odaklı modeller) ve parametrik olmayan ayar tekniklerinin (örneğin, LoRA ya da adapter tabanlı ince ayar) kullanılması hem bellek maliyeti hem de eğitim süresi açısından daha verimli çözümler sunabilir.

Son olarak, model çıktılarına ilişkin açıklanabilirlik ve güven değerlendirmeleri, bu tür sistemlerin gerçek dünyada benimsenmesi için kritik görünmektedir. Güven aralıkları, olasılık kalibrasyonu ve görsel hata analizleri gibi araçların entegrasyonu, geliştiricilerin model tahminlerini daha bilinçli şekilde yorumlamasına yardımcı olacaktır. Bu tezde gösterilen sonuçlar, hibrit ve şiddet duyarlı tahmin yaklaşımlarının umut verici olduğunu ortaya koymaktadır. Aynı zamanda, veri kalitesi, sınıf dengesizliği ve değerlendirme tasarımının gelecek çalışmalar için dikkatle ele alınması gereken unsurlar olduğunu vurgulamaktadır.

KAYNAKLAR

Alsolai, H., Roper, M. (2022). On the Use of Deep Learning in Software Defect Prediction: A Systematic Review. Journal of Systems and Software, cilt, sayı, sayfa aralığı.

Akimova, E. N., Bersenev, A. Y., Kuznetsov, S. O., ve diğ., 2021. A Survey on Software Defect Prediction Using Deep Learning. Mathematics, 9(11), 1180.

Albattah, W., Alenezi, M., Alqahtani, A., ve diğ., 2024. Software Defect Prediction Based on Machine Learning and Deep Learning Techniques: An Empirical Approach. AI, 5(4), 86.

Alsolai, H., Roper, M., 2022. On the Use of Deep Learning in Software Defect Prediction: A Systematic Review. Journal of Systems and Software, cilt, sayı, sayfa aralığı.

Arshad, M. A., Haque, M. M., Al-Ahmad, R., ve diğ., 2024. SevPredict: Exploring the Potential of Large Language Models in Software Maintenance. Computers, 13(4), 132.

Chidamber, S. R., Kemerer, C. F., 1994. A Metrics Suite for Object-Oriented Design. IEEE Transactions on Software Engineering, 20(6), 476–493.

Feng, Z., Guo, D., Tang, D., ve diğ., 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. Findings of the Association for Computational Linguistics: EMNLP 2020, 1536–1547.

Guo, Y., Gao, X., Jiang, B., 2024. An Empirical Study on JIT Defect Prediction Based on BERT-style Model. arXiv preprint, arXiv:2403.11158.

Jiang, Y., Shen, B., Gu, X., 2024. Just-In-Time Software Defect Prediction via Bi-modal Change Representation Learning. arXiv preprint, arXiv:2410.12107.

Lewis, M., Liu, Y., Goyal, N., ve diğ., 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 7871–7880.

Mashhadi, E., Ahmadvand, H., Hemmati, H., 2023. Method-Level Bug Severity Prediction Using Source Code Metrics and LLMs. arXiv preprint, arXiv:2309.03044.

McCabe, T. J., 1976. A Complexity Measure. IEEE Transactions on Software Engineering, 2(4), 308–320.

Pachouly, J., Ahirrao, A., 2022. A Systematic Literature Review on Software Defect Prediction Using Artificial Intelligence Techniques. Engineering Applications of Artificial Intelligence, 111, 104773.

Pan, C., Lu, M., Xu, B., 2021. An Empirical Study on Software Defect Prediction Using CodeBERT Model. Applied Sciences, 11(11), 4793.

Zhao, Y., Damevski, K., Chen, H., 2023. A Systematic Survey of Just-in-Time Software Defect Prediction. ACM Computing Surveys, 55(10), 1–35.

EKLER

EK A. Çalışmada Kullanılan Özniteliklerin Özet İstatistikleri

Bu ekte, tez kapsamında kullanılan eğitim veri seti ve test veri seti veri kümelerindeki sayısal özniteliklere ait özet istatistikler ile sınıf dağılımları sunulmuştur.

Çizelge A.1. Sayısal özniteliklerin özet istatistikleri

Öznitelik	Minimum	Maksimum	Ortalama	Standart Sapma
lc	-5,909	225,455	4,217	1,455
pi	-14,306	3,507	1,397	7,117
ma	-6,000	310000	4,814	2,032
nbd	-10000	235000	-365	1,030
ml	-4,167	138,33	3,442	1,163
d	-3,651	130,496	4,064	1,167
mi	-32,599	1,869	-102	7,151
fo	-5,000	292,500	4,946	1,704
r	-263	2,605	5,706	9,092
e	-1,628	1246452	1,942	8,088

Çizelge A.2. Hata şiddeti sınıflarının dağılımı

Sınıf	Sayı	Yüzde (%)
Düşük (0)	198	8,20
Orta (1)	1524	63,13
Yüksek (2)	203	8,41
Kritik (3)	489	20,26

EK B. Deneysel Kurulum ve Ortam Bilgileri

Bileşen	Değer
İşletim Sistemi	Ubuntu
Python sürümü	3
Kullanılan Donanım	CUDA
Çalışma ortamı	Jupyter Notebook

KİŞİSEL YAYIN VE ESERLER

Demir, R., Dervişoğlu, H., Halepmollası, R. ve Gündüz, H. (2025). Günlük Yazılım Geliştirme Akışlarında Büyük Dil Modelleri Kullanarak Hata Şiddeti Tahmini. *10. Uluslararası Mühendislik Bilimleri ve Multidisipliner Yaklaşımlar Kongresi*, İstanbul, 06-07 Aralık 2025.

ÖZGEÇMİŞ

İlk, orta ve lise öğrenimini Gaziantep'te tamamladı. 2017 yılında Kırklareli Üniversitesi Yazılım Mühendisliği bölümünde lisans öğrenimine başladı ve 2021 yılında mezun oldu. TÜBİTAK bünyesinde görev yapmaktadır. 2022 yılında Kocaeli Üniversitesi Fen Bilimleri Enstitüsü Yazılım Mühendisliği Ana Bilim Dalı'nda Yüksek Lisans Programına başladı. 2026 yılında mezun olma durumundadır.